



FLOORPLANMASTER: BLENDER ADD-ON PRO PARAMETRICKÉ MODELOVÁNÍ MÍSTNOSTÍ

Oskar Wladař

Bakalářská práce
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Studijní program: Informatika
Specializace: Počítačová Grafika
Vedoucí: Ing. Lukáš Bařinka
May 05, 2026

České vysoké učení technické v Praze

Fakulta informačních technologií

© 2026 Oskar Wladař. Všechna práva vyhrazena.

Tato práce vznikla jako školní dílo na Českém vysokém učení technickém v Praze, Fakultě informačních technologií. Práce je chráněna právními předpisy a mezinárodními úmluvami o právu autorském a právech souvisejících s právem autorským. K jejímu užití, s výjimkou bezúplatných zákonných licencí a nad rámec oprávnění uvedených v Prohlášení, je nezbytný souhlas autora.

Odkaz na tuto práci: Oskar Wladař. *FloorPlanMaster: Blender add-on pro parametrické modelování místností*. Bakalářská práce. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

TODO: Poděkování

Prohlášení

TODO

V Prague dne May 05, 2026

Abstrakt

Práce se zabývá návrhem a implementací rozšiřovacího modulu FloorPlanMaster pro Blender umožňujícího parametrické a nedestruktivní modelování architektonických půdorysů. Na základě analýzy tří cílových skupin uživatelů — architektů, 3D vizualizátorů a game designérů — a komparativní analýzy existujících nástrojů byly stanoveny funkční a nefunkční požadavky. Pro jejich splnění byla navržena třívrstvá hybridní architektura kombinující strukturální graf pro topologii stěn a styků, sémantický graf místností pro automatickou detekci uzavřených cyklů a synchronizační vrstvu přenášející data do Blender meshe s pojmenovanými atributy čtenými modulem Geometry Nodes.

Výsledkem je funkční add-on implementovaný v Pythonu, zahrnující interaktivní nástroj pro kreslení stěn na základě modálního operátoru, parametrickou editaci tloušťky a výšky stěn, automatickou detekci místností s výpočtem jejich ploch a podporu otvorů (dveře, okna) s poziční validací. Addon zaplňuje identifikovanou mezeru v ekosystému programu Blender a umožňuje celý pracovní postup — od prvního náčrtu dispozice přes parametrické úpravy až po finální mesh — realizovat v jednom prostředí bez nutnosti přepínat mezi specializovanými programy.

Klíčová slova Blender add-on, parametrické modelování, půdorys, architektonická vizualizace, Geometry Nodes, grafový datový model, prostorová dispozice, Python API

Abstract

The thesis deals with the design and implementation of the FloorPlanMaster extension for Blender, enabling parametric and non-destructive modelling of architectural floor plans. Based on an analysis of three target user groups — architects, 3D visualizers, and game designers — and a comparative analysis of existing tools, functional and non-functional requirements were elicited. To meet these requirements, a three-layer hybrid architecture was designed, combining a structural graph for wall and junction topology, a semantic room graph for automatic detection of closed cycles, and a synchronization layer transferring data into the Blender mesh with named attributes read by the Geometry Nodes module.

The result is a functional add-on implemented in Python, encompassing an interactive wall drawing tool based on a modal operator, parametric editing of wall thickness and height, automatic room detection with area computation, and support for openings (doors and windows) with positional validation. The add-on fills an identified gap in the Blender ecosystem and allows the entire workflow — from the initial floor plan sketch through parametric editing to the final mesh — to be carried out within a single environment, without the need to switch between specialized applications.

Keywords Blender add-on, parametric modeling, floor plan, architectural visualization, Geometry Nodes, graph data model, spatial layout, Python API

Obsah

Úvod	1
1 Analýza	2
1.1 Doménová analýza	2
1.1.1 Parametrické modelování a prostorová dispozice	3
1.1.2 Analýza existujících řešení	3
1.2 Cílové skupiny	7
1.2.1 Uživatelské skupiny	7
1.2.2 Persony	9
1.3 Vstupy a výstupy	10
1.3.1 Vstupy	10
1.3.2 Výstupy	11
1.4 Scénáře použití	11
1.4.1 UC 1.1: Hmotová studie na základě stavebního programu	11
1.4.2 UC 1.2: Kreslení dispozice tužkou	11
1.4.3 UC 1.3: Kontrola rozměrů vůči normovým minimům	12
1.4.4 UC 2.1: Obkreslení dodaného 2D půdorysu	12
1.4.5 UC 2.2: Příprava modelu pro renderovací pipeline	12
1.4.6 UC 2.3: Rychlá editace vlastností prvků přes kontextovou nabídku	12
1.4.7 UC 3.1: Rychlý level blackout	13
1.4.8 UC 3.2: Finalizace a export herní úrovně	13
1.4.9 UC 3.3: Interaktivní úprava rozložení místností	13
1.5 Analýza požadavků	13
1.5.1 Funkční požadavky	14
1.5.2 Nefunkční požadavky	15
1.5.3 Prioritizace požadavků	16
1.6 Technická analýza	18
1.6.1 Architektura Blenderu	18
1.6.2 Interaktivní kreslení a interakce ve viewportu	18
1.6.3 Reprezentace geometrie	20
1.6.4 Datový model	23
1.6.5 Tvorba otvorů pro okna a dveře	25
1.6.6 Ukládání dat a správa metadat	26
1.6.7 Uživatelské rozhraní	28
1.6.8 Finalizační nástroj	29
2 Návrh	31
2.1 Definice rozsahu MVP	31
2.2 Architektura systému	32

2.2.1	Proč oddělená sémantická vrstva	33
2.2.2	Třívrstvá hybridní architektura	34
2.2.3	Vzor MVC v kontextu Blenderu	34
2.2.4	Principy návrhu	36
2.2.5	Rozšiřitelnost architektury	36
2.3	Datový model	36
2.3.1	Vrstva 1: Strukturální graf	36
2.3.2	Vrstva 2: Sémantický graf místností	38
2.3.3	Vrstva 3: Synchronizační bridge	39
2.3.4	Vztah mezi vrstvami	40
2.4	Návrh funkcí	41
2.4.1	FP1 — Nástroj tužka	41
2.4.2	FP2 — Parametrické objekty a otvory	43
2.4.3	FP3 — Detekce místností a metadata	44
2.4.4	FP4 — Finalizační nástroj	44
2.4.5	FP5 — Kontextová nabídka	46
2.4.6	FP6 — Interaktivní manipulátory	46
2.4.7	FP7 — Automatické kótování	46
2.5	Návrh uživatelského rozhraní	47
2.5.1	Toolbar — umístění nástroje tužka	47
2.5.2	Postranní panel (N-panel)	47
2.5.3	Klávesové zkratky	50
2.5.4	FloorPlan kontext — simulovaný pracovní mód	52
2.6	Testování a validace návrhu	53
2.6.1	Pokrytí požadavků	53
2.6.2	Průchod scénáři použití	55
2.7	Hodnocení návrhu uživatelského rozhraní	56
2.7.1	Konzistence s ekosystémem Blenderu	56
2.7.2	Heuristické hodnocení	56
2.7.3	Kognitivní průchody	57
2.7.4	Závěr hodnocení	58
3	Implementace	59
3.1	Addon pro Blender	59
3.2	Organizace modulů	60
3.3	Vrstvy	60
3.3.1	Vrstva 1 — Strukturální graf	61
3.3.2	Vrstva 2 — Graf místností	62
3.3.3	Vrstva 3 — Synchronizace s Blenderem	62
3.3.4	Geometry Nodes — generování 3D geometrie	63
3.4	Funkce	64
3.4.1	FP1 — Nástroj tužka	64

3.4.2	FP2 — Výběr a parametrické úpravy	65
3.4.3	FP3 — Metadata místností	65
3.4.4	FP4 — Finalizace a bake	65
3.4.5	Neimplementované funkce	66
3.5	Uživatelské rozhraní	66
3.5.1	N-panel	66
3.5.2	Overlay manager	66
3.5.3	Sdílený stav výběru	67
3.6	Současná omezení implementace	68
4	Testování	70
	Závěr	71
	Bibliografie	72
	Contents of the attachment	73

Seznam výpisů kódu

Seznam tabulek

Tabulka 1	Srovnání analyzovaných nástrojů podle hodnotících kritérií	6
Tabulka 2	Mapování funkčních balíčků na scénáře použití (● = relevantní)	16
Tabulka 3	Vážená prioritizace požadavků podle cílových skupin	17
Tabulka 4	Návratové hodnoty modálního operátoru	19
Tabulka 5	Srovnání přístupů k reprezentaci geometrie stěn	22
Tabulka 6	Srovnání metod pro tvorbu otvorů ve stěnách	26
Tabulka 7	Přístupy k perzistenci grafových dat v Blenderu	27
Tabulka 8	MoSCoW prioritizace prvků FP1–FP7; must-have prvky tvoří jádro MVP	32
Tabulka 9	Alternativy reprezentace sémantiky v Blender addonu a jejich trade-offy	33
Tabulka 10	Pojmenované atributy Vrstvy 3 zapisované synchronizačním modulem a čtené Geometry Nodes	40
Tabulka 11	Přehled globálních nastavení v sekci Nastavení N-panelu s výchozími hodnotami	49
Tabulka 12	Klávesové zkratky addonu s kontextem a zdůvodněním volby klávesy	51
Tabulka 13	Tabulka pokrytí požadavků návrhovou dokumentací; ✓ = součást MVP, — = odloženo za MVP	54

Seznam obrázků

Obrázek 1	MVC diagram reprezentující oddělení vrstev addonu	35
Obrázek 2	Diagram tříd na vrstvě 1	37
Obrázek 3	Diagram tříd na vrstvě 2	38
Obrázek 4	Diagram tříd na vrstvě 3	39
Obrázek 5	Statický pohled — závislosti tříd	41
Obrázek 6	Dynamický pohled — sekvenční diagram	41
Obrázek 7	Stavový diagram nástroje tužka	42
Obrázek 8	Stavový diagram finalizačního nástroje	45
Obrázek 9	Návrh UI pro sekci Nástroje	48

Obrázek 10	Návrh UI pro sekci Místnosti	48
Obrázek 11	Návrh UI pro sekci Nastavení	49

Seznam zkratek

- API* – Application Programming Interface [4](#) [39](#) [47](#) [52](#) [56](#) [59](#)
BIM – Building Information Modeling [2](#)
BLF – Blender Font Library [15](#)
CAD – Computer-Aided Design [2](#) [43](#)
DNA – Data Architecture (Blender's internal C-structure layer) [18](#)
DWG – Drawing — AutoCAD native binary format [10](#)
DXF – Drawing Exchange Format [10](#) [32](#)
FBX – Filmbox — Autodesk proprietary 3D interchange format [11](#)
GIL – Global Interpreter Lock [22](#)
GN – Geometry Nodes [21](#) [59](#) [60](#) [62](#) [63](#) [63](#)
GPU – Graphics Processing Unit [15](#) [32](#) [35](#) [42](#) [66](#)
HUD – Heads-Up Display [29](#) [35](#) [42](#) [43](#)
IFC – Industry Foundation Classes [4](#) [32](#)
ISO – International Organization for Standardization [5](#)
JSON – JavaScript Object Notation [27](#)
MVC – Model-View-Controller [18](#) [34](#) [64](#)
MVP – Minimum Viable Product [31](#) [59](#) [68](#) [68](#)
OBJ – Wavefront OBJ — open 3D geometry file format [11](#)
PDF – Portable Document Format [7](#)
RMB – Right Mouse Button [29](#) [46](#)
RNA – Runtime Notification Architecture (Blender's Python-accessible reflection layer) [18](#)
SVG – Scalable Vector Graphics [4](#)
UI – User Interface [16](#) [47](#) [56](#) [59](#) [66](#)
UV – UV coordinates — 2D texture mapping coordinates [12](#)
UX – User Experience [17](#)

Úvod

Bakalářská práce se zabývá návrhem a implementací rozšiřovacího modulu **FloorPlanMaster** pro 3D software Blender. Cílem je navrhnout a implementovat nástroj, který do Blenderu přináší parametrické, nedestruktivní modelování prostorových dispozic — umožňuje interaktivně kreslit půdorysy, definovat místnosti a upravovat rozměry jednotlivých prvků bez nutnosti ručně přestavovat geometrii.

Blender je výkonný, volně dostupný 3D software s aktivní komunitou vývojářů rozšiřovacích modulů. Přestože dnes slouží i pro komplexní architektonické projekty, pro tvorbu půdorysů a prostorových dispozic mu chybí specializované nástroje. Existující rozšíření tento nedostatek adresují jen částečně — obvykle buď postrádají skutečně nedestruktivní přístup, nebo jsou příliš složité pro ranou fázi konceptuálního návrhu. Výsledkem je mezera, kterou FloorPlanMaster cílí zaplnit. Addon je určen zejména architektům a vizualizátorům, kteří potřebují rychle skicovat dispozice v prostředí, které již znají, aniž by přecházeli do specializovaného CAD nebo BIM softwaru.

Řešení je navrženo jako vícevrstvá architektura, která odděluje logiku půdorysu od jeho vizualizace. Datový model půdorysu je spravován čistě v Pythonu a komunikuje s Blenderem jednosměrně — změna parametru se okamžitě promítne do 3D zobrazení, aniž by bylo nutné jakkoliv zasahovat do výsledné geometrie. Addon je implementován jako rozšiřující modul pro Blender napsaný v Pythonu a nevyžaduje žádný externí software.

Struktura práce postupuje od obecného kontextu k technickým detailům. Analytická kapitola vymezuje problémovou doménu, definuje cílové skupiny uživatelů a odvozuje požadavky na addon. Kapitola návrhu překládá tyto požadavky do softwarové architektury a návrhu funkcí. Následují kapitoly věnované implementaci, testování a závěrečnému zhodnocení dosažených výsledků.

[illegible]

Analýza

Blender je všestranný, ale pro architektonické skicování nevhodně vybavený nástroj — každá změna půdorysu se mění v destruktivní opravu vrcholů a přepočítávání otvorů, což narušuje plynulost návrhového procesu. Aby bylo možné FloorPlanMaster navrhnout správně, je nutné nejprve porozumět tomu, proč stávající řešení nestačí, kdo addon skutečně bude používat a co od něj v konkrétních situacích čeká. Tato analýza tedy postupně buduje obraz od problémové domény přes uživatele a jejich scénáře až ke strukturovaným požadavkům a výběru technologií, na nichž addon stojí.

1.1 Doménová analýza

Architektonická dispozice vzniká iterativně: místnosti se posouvají, chodby se zužují, stěny mění tloušťku — a každá z těchto změn by měla být otázkou sekund, ne minut ručního přepočítávání geometrie. Cílem doménové analýzy je pochopit, proč Blender jako obecný nástroj tento požadavek nativně nenaplnuje a jaký typ řešení by mezeru zaplnil. Nejprve je proto potřeba vymezit, čím se parametrické modelování liší od klasické polygonální práce, a poté zmapovat, jak se s tímto problémem vypořádala stávající řešení — jak uvnitř programu Blender, tak mimo něj.

Blender [1] je primárně 3D polygonální modelovací nástroj, nikoliv specializovaný CAD nebo BIM software. Nabízí obrovskou flexibilitu, pro specifické potřeby architektonického navrhování však postrádá nativní nástroje, což vede k neefektivním a zdoluhavým pracovním postupům. Tvorba půdorysů a 3D dispozic budov je v čistém Blenderu zdoluhavá a destruktivní — každá změna vyžaduje manuální opravu okolní geometrie, posouvání vrcholů a přepočítávání otvorů.

Cílem projektu je tedy vytvoření rozšiřovacího modulu pro Blender zaměřeného na parametrické modelování prostorových dispozic. Uživatel bude moci prostorové dispozice intuitivně vytvářet a nedestruktivně upravovat pomocí parametrů; primárním cílem je urychlit ranou fázi architektonického návrhu.

1.1.1 Parametrické modelování a prostorová dispozice

Parametrické modelování je způsob vytváření 3D modelů, kde tvar objektu není definován pevně, ale pomocí proměnných parametrů (číselných hodnot) a geometrických vztahů. Tvůrce přímo kontroluje vstupy a algoritmickou logiku závislostí: čtverec lze například nadefinovat parametrem šířky, parametrem délky a pravidlem kolmosti stran, přičemž pozdější změna parametrů způsobí automatické překreslení tvaru bez nutnosti manuálního zásahu.

Oproti klasickému polygonálnímu modelování, kde se pracuje s body, hranami a plochami jejich přímou manipulací, parametrické modelování pracuje s čísly, funkcemi a historií kroků. Polygonální modelování je primárně vhodné pro hry, filmy a animace; parametrické modelování převládá v architektuře, strojírenství a produktovém designu.

Rozlišují se dvě hlavní paradigmatu [2], [3]. *Historické* modelování, typické pro CAD a BIM software, si pamatuje časovou osu úprav: software zaznamenává sekvenci kroků a umožňuje vrátit se k dřívějšímu kroku a automaticky přepočítat všechny závislé operace. Toto paradigma je standardem v průmyslovém CAD (SolidWorks) i architektonickém BIM (Revit). *Algoritmické* modelování, označované také jako vizuální skriptování, popisuje geometrii pomocí uzlových grafů a je analogické Geometry Nodes v Blenderu.

Parametrické modelování v architektuře přináší posun od tradičního reprezentativního kreslení k algoritmickému přístupu. Tradiční CAD systémy se spoléhají na explicitní definici geometrie pomocí statických bodů a úseček. Parametrický přístup zavádí systém vzájemně propojených proměnných, matematických omezení a deduktivních pravidel, která dynamicky generují a aktualizují výslednou formu. Úprava jediného parametru — například celkové výšky podlaží nebo tloušťky nosné stěny — automaticky a kaskádovitě modifikuje všechny závislé entity.

Prostorová dispozice je logické a funkční uspořádání trojrozměrného objektu do smysluplných celků (místností a zón) s definovanými vztahy mezi nimi. Tento pojem vymezuje konkrétní předmět návrhu, se kterým addon pracuje.

1.1.2 Analýza existujících řešení

Před návrhem nového nástroje je nutné pochopit, co již existuje a kde existující řešení selhávají — jinak hrozí, že FloorPlanMaster jen zopakuje jejich slabiny. Tato sekce proto hodnotí nejvýraznější architektonické addony přímo pro Blender a porovnává je s nástroji mimo jeho ekosystém, aby bylo jasné, jaká mezera zůstala nezaplněna.

Všechny nástroje jsou hodnoceny podle pěti shodných kritérií: (1) *interaktivní kreslení půdorysu* — přímé klikání bodů do viewportu způsobem tužky; (2) *parametrická editace stěn* — možnost kdykoli změnit tloušťku, výšku nebo polohu stěny bez narušení sousední geometrie; (3) *automatická detekce místností* — samostatné rozpoznávání uzavřených cyklů stěn jako místností s výpočtem ploch; (4) *správa otvorů* — okna a dveře svázané s parametry stěny, pohybující se spolu s ní; a (5) *nedestruktivní workflow* — parametrická úprava geometrii okamžitě přepočítá bez ručního zásahu do okolní topologie.

1.1.2.1 Architektonické rozšiřující moduly pro Blender

Blender za posledních deset let prošel dramatickou evolucí. Původně vnímaný jako nástroj pro organické modelování, animace a vizuální efekty byl díky open-source modelu a robustnímu Python [API](#) transformován v platformu schopnou realizovat komplexní architektonické projekty. Vzniklo tak několik specializovaných addonů.

Archimesh [\[4\]](#) je základním kamenem architektonických nástrojů pro Blender. Vytvořil ho Antonio Vazquez s cílem automatizovat tvorbu interiérových a exteriérových prvků, která by jinak zabrala hodně času manuálním modelováním. Díky stabilitě a užitečnosti byl dlouhodobě integrován přímo do oficiální distribuce Blenderu jako komunitní doplněk a je primárně určen pro rychlé skicování prostor a interiérový design.

Pro tvorbu místností nabízí Archimesh dva přístupy: definování počtu stěn a jejich rozměrů, nebo využití nástroje Grease Pencil, kde uživatel v půdorysném pohledu nakreslí hrubé obrysy místností a doplněk tyto tahy automaticky převede na trojrozměrné stěny. Addon dále umožňuje automatické generování podlah a stropů. Co se týče otvorů, podporuje dva typy oken — kolejnicová a panelová — s parametrickými parapety a systémem žaluzií. Součástí je rovněž generátor kuchyňských linek, polic a dalšího interiérového vybavení [\[4\]](#).

Archipack [\[5\]](#) byl vytvořen jako robustnější a výkonnější alternativa k Archimesh, zaměřená primárně na profesionální architekty a vizualizátory. Autorem je Stephen Leger a addon existuje ve dvou verzích — Community Edition a Pro. Klíčovým rysem je důraz na interaktivní manipulaci: systém Auto-manipulate on select při výběru objektu zobrazí přímo ve 3D viewportu táhla (gizmos) a textové popisky. Parametry prvků jsou spravovány vlastním systémem vlastností, které zůstávají zachovány po celou dobu životnosti projektu — uživatel může kdykoliv vybrat stěnu a změnit její tloušťku nebo výšku a všechny připojené prvky automaticky na tuto změnu reagují. Pro export nabízí Archipack generování řezů a půdorysů ve formátu [SVG](#). Pro verze pak export do formátu [IFC](#) [\[5\]](#).

BonsaiBIM [\[6\]](#) zaujímá zcela odlišnou pozici: na rozdíl od Archimesh a Archipack je navržen jako nativní platforma pro tvorbu a správu informačních

modelů budov, fungující na mezinárodním standardu IFC [7] (ISO 16739). Zatímco Archimesh a Archipack jsou nadstavby nad standardním modelovacím procesem a jejich data jsou spjata s `.blend` souborem, BonsaiBIM transformuje program Blender v plnohodnotný prohlížeč a editor databáze IFC [6].

1.1.2.2 Architektonické nástroje mimo Blender

Mimo ekosystém programu Blender existují tři dominantní nástroje pro architektonické modelování, která tvoří referenční rámec pro analýzu potřeb cílových skupin. **SketchUp** [8] je postaven na principu přímého modelování ploch a jako prioritu klade rychlost a intuitivní transformaci myšlenky do 3D formy. **AutoCAD** [9] představuje standard pro 2D dokumentaci a detailní rýsování — funguje jako digitální rýsovací prkno, nepostradatelné pro technické výkresy. **Revit** [10] je robustní parametrický BIM nástroj, kde každý prvek v modelu je instancí v databázi s definovanými funkčními vztahy; změna v jednom zobrazení se automaticky promítne do všech výkresů a výkazů.

■ **Tabulka 1** Srovnání analyzovaných nástrojů podle hodnotících kritérií

(1) Kreslení nástroj	(2) Param. editace	(3) Místnosti	(4) Otvory	(5) Nede- struktivní
Archimesh [4]	Částečně	Ne	Ano	Částečně
Archipack [5]	Ano	Ne	Ano	Ano
Bonsai-BIM [6]	Ano	Ne	Ano	Ano
SketchUp [8]	Částečně	Ne	Částečně	Ne
Autodesk [9]	Ne	Ne	Základní	Ne
Revit [10]	Ano	Ano	Ano	Ano

1.1.2.3 Identifikované mezery a příležitosti

Přehled existujících řešení odhaluje opakující se nedostatky, které představují hlavní příležitosti pro budoucí návrh modulu FloorPlanMaster.

Chybějící nástroj pro přirozené kreslení půdorysů. Ani Archimesh, ani Archipack nenabízejí plnohodnotný nástroj pro skicování dispozice klikáním bodů přímo ve viewportu způsobem srovnatelným se SketchUpem nebo AutoCADem. Archimesh sice umí interpretovat tahy Grease Pencil jako stěny a Archipack vygeneruje stěny z křivky, ale ani v jednom případě nevzniká dojem přirozené „tužky v půdorysu“. FloorPlanMaster proto může zavést interaktivní Pencil Tool jako primární vstupní metodu — modální operátor, který by sledoval kurzor a při každém kliknutí okamžitě vytvořil stěnu.

Nespolehlivá správa otvorů. Systém automatických otvorů v modulu Archimesh má dokumentované slabiny u složitějších stěn a při použití solveru Exact. Otvory v Archipacku sice fungují robustně, ale vložení okna nebo dveří často vede k nepřehlednému zanořování do složitých panelů nastavení. Zde se nabízí prostor pro záměrné zjednodušení rozhraní: otvor by se dal přidávat pouhým výběrem stěny a zadáním tří základních hodnot (šířka, výška, výška parapetu), zatímco o geometrickou konzistenci by se staral modifikátor založený na Geometry Nodes (Mesh Boolean) propojený datovou vazbou.

Chybějící přehled o místnostech a plochách. Žádný z analyzovaných addonů pro Blender nedetekuje uzavřené cykly stěn jako místnosti automaticky a nezobrazuje jejich plochu jako primární informaci. Analytické výměry jsou v Archipacku i Archimesh skryté v technickém rozhraní, nebo zcela chybí. Přitom právě integrace detekce místností a zobrazení ploch přímo do hlavního přehledu v N-panelu by představovala klíčový výstup jak pro architekta, tak pro game designéra při ověřování parametrů návrhu.

Izolace externích nástrojů mimo ekosystém programu Blender. SketchUp, AutoCAD a Revit sice ve svém oboru vynikají, ale postrádají integraci do programu Blender — přechod z konceptuálního modelu z programu SketchUp do plnohodnotné renderovací scény v programu Blender vyžaduje export, čištění topologie a vede ke ztrátě parametrických vlastností. Cílem FloorPlanMasteru je tuto bariéru eliminovat a zajistit, aby celý životní cyklus — od prvního načrtnutí dispozice přes parametrické úpravy až po finalizovanou síť (mesh) pro rendering — probíhal přímo v jedné scéně v programu Blender.

1.2 Cílové skupiny

Parametrické modelování půdorysů v Blenderu může urychlit práci architektovi skicujícím varianty pro klienta, vizualizátorce převádějícím 2D výkresy z [PDF](#) do 3D prostoru, i game designérovi, který po sérii testování potřebuje narychlo rozšířit herní chodbu. Každý z nich však od addonu očekává něco trochu jiného. Cílem této sekce je tyto skupiny přesně vymezit, pojmenovat jejich konkrétní frustrace s nativním Blenderem a ztělesnit je v uživatelských personách. Ty pak poslouží jako měřítko pro hodnocení každého budoucího návrhového rozhodnutí.

1.2.1 Uživatelské skupiny

Společným problémem všech tří skupin je frustrace z toho, že Blender nedokáže udržet krok s rychlostí jejich myšlení: potřeba změnit dispozici často přichází ve chvíli, kdy je geometrie již pevně spojená v jeden celek a její

úprava je zdoluhavá. Každá skupina však naráží na specifické problémy a upřednostňuje jiné vlastnosti addonu, proto je nutné popsat je zvlášť.

1.2.1.1 1. Architekti ve fázi konceptuálního navrhování

Architekti pracující na konceptuálním návrhu tvoří primární cílovou skupinu addonu. Jedná se o profesionály nebo studenty tvořící úvodní hmotové a prostorové studie. V této fázi se nezdržují mikrodetaily (jako je typ kování u dveří), ale zaměřují se na celkové proporce, návaznosti místností a celkové uspořádání půdorysu.

Vyžadují rychlou iteraci návrhu, přesné zadávání číselných hodnot (délky, šířky, výšky) a nedestruktivní úpravy — tedy možnost kdykoliv změnit parametry místnosti nebo plynule posouvat okna a dveře podél stěn. Modelování těchto prvků pomocí standardních nástrojů Blenderu pro ně představuje destruktivní proces: každá změna vyžaduje manuální opravu okolní geometrie, posouvání vrcholů a složité přepočítávání otvorů. To odvádí jejich pozornost od samotného navrhování a narušuje plynulost tvůrčí práce.

1.2.1.2 2. 3D umělci a vizualizátoři

Vizualizátoři potřebují co nejrychleji postavit základní geometrii místností, aby se mohli naplno věnovat tomu hlavnímu — materiálům, nasvícení a samotnému renderingu. Jde o tvůrce zaměřené primárně na estetiku. Hrubá stavba pro ně představuje pouhé „plátno“, do kterého následně vkládají detailní modely nábytku a vybavení.

Kromě rychlosti je pro ně klíčová také čistá topologie výsledné geometrie, na které budou bezchybně fungovat textury a další renderovací modifikátory. Ruční modelování stěn a vyřezávání oken pomocí nativních Boolean modifikátorů totiž často vytváří nevzhlednou topologii a její ruční čištění je pro tyto umělce nepříjemnou a vysoce neefektivní rutinou.

1.2.1.3 3. Game a level designéři

Game a level designéři využívají addon k rychlé tvorbě a iteraci herních blockoutů — hrubých modelů úrovní sloužících k testování hratelnosti, průchodnosti a pohybu hráče či kamery.

Požadují především modulární přístup k tvorbě prostoru, schopnost okamžitě měnit proporce na základě zpětné vazby z herního testování a bezproblémový export hotových tvarů do enginů jako Unreal Engine nebo Unity. Nativní modelovací nástroje Blenderu nejsou pro rychlý level design optimalizovány, a tak je ruční upravování rozměrů místností nebo přesouvání otvorů během prototypování příliš těžkopádné.

1.2.2 Persony

Abstraktní popis cílových skupin neukáže, jak přesně se zmíněné frustrace projevují v praxi — zda architekta brzdí přepočítávání návazných stěn, nebo spíše chybějící přehled o podlahové ploše; zda vizualizátorku zdržuje samotné modelování, nebo až následné čištění topologie po ořezu. Pro každou skupinu je proto definována jedna konkrétní persona. Její profil, cíle a frustrace slouží jako referenční bod při rozhodování o podobě a funkčnosti addonu.

1.2.2.1 1. Architekt Adam

Adam (32 let) pracuje v menším architektonickém ateliéru a má na starosti úvodní studie a komunikaci s klienty při hledání tvaru a dispozice budovy. Běžně pracuje v profesionálních CAD a BIM programech (AutoCAD, Revit), ale pro rychlé 3D koncepty a objemové studie si oblíbil Blender díky jeho svižnosti a real-time zobrazení. Neumí však programovat a práce s Geometry Nodes je pro něj příliš složitá.

Jeho typickým cílem je během pár hodin vytvořit pro klienta tři různé varianty prostorového uspořádání domu. Primárně ho zajímá hmota, návaznosti místností a základní rozměry. Frustruje ho zdlouhavé extrudování polygonů v nativním Blenderu: když klient požádá o rozšíření obývacího pokoje o metr, Adam musí ručně posouvat vertexy a složitě přepočítávat navazující stěny, přičemž mu navíc chybí okamžitý přehled o rozměrech místností. Od addonu očekává jednoduché rozhraní, ve kterém zadá rozměry místnosti nebo je naskicuje, a systém při jakékoliv změně automaticky zachová tloušťku zdiva a nerozbije návaznost na další prostory.

1.2.2.2 2. Vizualizátorka Věra

Věra (28 let) je vizualizátorka na volné noze, která se specializuje na tvorbu fotorealistických interiérů pro developery a realitní kanceláře. Blender ovládá na špičkové úrovni — má hluboké znalosti materiálů, nasvícení i renderingu — a profiluje se spíše umělecky než technicky.

Běžně dostává od klienta 2D půdorys v PDF a potřebuje z něj co nejrychleji vytvořit hrubý 3D obraz bytu, aby se mohla věnovat tomu hlavnímu: světlu, texturám a vybavení. Zdlouhavé modelování holých stěn ji zdržuje a odvádí od kreativní práce. Navíc nativní vyřezávání otvorů přes Boolean jí často zaneřádí síť ošklivou topologií, kterou pak musí před renderingem složitě čistit. Očekává proto nástroj podobný tužce, kterým podložený půdorys jednoduše obkreslí, a možnost vkládat otvory na jedno kliknutí s jistotou, že addon udrží topologii čistou.

1.2.2.3 3. Level designer Denis

Denis (25 let) je vývojář v nezávislém herním studiu. Navrhuje herní úrovně a testuje pohyb hráče v prostoru. Primárně pracuje v herních enginech (Unreal Engine, Unity), přičemž Blender využívá k rychlé tvorbě takzvaných blockoutů — hrubé geometrie určené pro okamžité testování hratelnosti.

Jeho úkolem je v krátkém čase vybudovat rozsáhlou herní mapu (například spleť chodeb a místností), vyexportovat ji do engineu a projít si ji s herní postavou. Když při testování zjistí, že je chodba příliš úzká nebo strop příliš nízký, je úprava takové úrovně v čistém Blenderu (pokud je spojená do jednoho souvislého kusu geometrie) neefektivní a zdlouhavá. Od addonu proto vyžaduje robustnost a rychlost iterace: parametrický přístup by mu měl umožnit kliknout na chodbu, v panelu přepsat její šířku ze dvou metrů na tři, a nechat systém automaticky vyřešit zbytek. Nezbytností je pro něj také export, který po přesunu do engineu nerozbije fyzikální kolize.

1.3 Vstupy a výstupy

FloorPlanMaster je navržen tak, aby dokázal reagovat na reálné pracovní situace: architekt často začíná s prázdnou scénou a úvodní myšlenkou, vizualizátorka dostane od klienta 2D půdorys ve formátu PDF a game designér vychází z přibližných rozměrů v herním design dokumentu. Tato sekce jasně vymezuje, jaké formy vstupních dat addon zpracovává a jaké výstupy následně produkuje, čímž definuje hranice životního cyklu celého modelu.

1.3.1 Vstupy

Addon dokáže zpracovat tři základní kategorie vstupů. První kategorií jsou **2D podklady** — naskenované ruční skice, výkresy v PDF, obrázky půdorysů nebo importované 2D CAD výkresy (**DXF/DWG**), které uživatel potřebuje převést do 3D prostoru. Podkladový soubor se typicky umístí na pozadí scény a slouží jako vizuální reference pro následné obkreslování.

Druhou kategorií je **kvantitativní zadání** — přesný seznam požadavků od klienta nebo technické specifikace (např. „obývací pokoj musí mít minimálně 30 m²“ nebo „minimální šířka chodby jsou 2 metry“). Tyto hodnoty lze přímo zadávat do panelu nástroje jako číselné parametry.

Třetí kategorií je **volný návrh** — situace, kdy uživatel nemá žádné přesné podklady, začíná s prázdnou scénou a potřebuje nástroj, který ho nebude omezovat v rychlém a intuitivním skicování úvodních konceptů.

1.3.2 Výstupy

Výstupy z addonu se dělí do tří kategorií. Tou první je **3D hmotový model** — prostorová 3D reprezentace stěn, místností a otvorů, která slouží primárně k vizuální kontrole proporcí, tvorbě hmotových renderů, analýze osvětlení nebo k základní prezentaci klientovi.

Druhou kategorií je **optimalizovaná geometrie pro export**. Jedná se o čistou topologickou síť (mesh) bez chyb nebo překrývajících se stěn, kterou může level designér okamžitě a bez dalších úprav vyexportovat (například do formátu **FBX** nebo **OBJ**) pro použití v herním enginu.

Třetí a poslední kategorií jsou **prostorová a analytická data** — rychlá vizuální zpětná vazba pro uživatele. Zahrnuje automatické výpočty podlahové plochy jednotlivých místností a jasnou indikaci tloušťky stěn přímo v uživatelském rozhraní.

1.4 Scénáře použití

Abstraktní požadavky typu „parametrické úpravy“ či „nedestruktivní workflow“ samy o sobě nedefinují, s jakou odezvou se musí stěna přepočítat po kliknutí myši, ani jak se systém zachová, když uživatel do scény přiloží PDF výkres a začne jej obkreslovat. Tyto situace konkretizují až scénáře použití (Use Cases). Každý scénář sleduje konkrétní personu od jejího prvního kliknutí až po požadovaný výsledek a jasně odkrývá, které funkce modulu jsou pro daný úkol klíčové.

1.4.1 UC 1.1: Hmotová studie na základě stavebního programu

Architekt zadá do panelu nástroje požadovanou podlahovou plochu a poměr stran pro každou místnost (například 30 m², poměr 1:1,5). Addon automaticky vypočítá potřebné rozměry a vloží pravoúhlou místnost přímo do scény. Uživatel tento postup zopakuje pro všechny místnosti ze stavebního programu. Výsledkem je schematická dispozice, u níž lze v panelu okamžitě ověřit plochu každého prostoru.

1.4.2 UC 1.2: Kreslení dispozice tužkou

Architekt aktivuje kreslicí nástroj a pouhým klikáním bodů přímo ve 3D viewportu načrtne hrubou dispozici. Addon průběžně generuje stěny a při uzavření polygonu automaticky detekuje vzniklé místnosti. Uživatel následně doladí tloušťku a výšku stěn zadáním přesných hodnot v N-panelu.

1.4.3 UC 1.3: Kontrola rozměrů vůči normovým minimům

Architekt má navrženou dispozici a potřebuje ověřit, zda šířky chodeb a rozměry místností splňují minimální normové požadavky. V N-panelu zapne kótovací vrstvu (overlay). Addon prostřednictvím překreslovacího modulu (BLF draw handler) okamžitě zobrazí délky všech stěn a plochy místností přímo ve viewportu. Uživatel tak může vizuálně zkontrolovat kritická místa a případně upravit parametry stěn v panelu.

1.4.4 UC 2.1: Obkreslení dodaného 2D půdorysu

Vizualizátorka si na pozadí scény vloží referenční obrázek s půdorysem. Aktivuje kreslicí nástroj a se zapnutým přichytáváním (snapping) na existující uzly odklikává rohy místností přesně podle podkladu. Addon během kreslení průběžně generuje stěny a při uzavření cyklů automaticky detekuje jednotlivé místnosti. Výšku a tloušťku stěn následně hromadně nebo individuálně nastaví v N-panelu.

1.4.5 UC 2.2: Příprava modelu pro renderovací pipeline

Vizualizátorka vybere na existujícím půdorysu konkrétní stěnu a v N-panelu k ní přidá otvor zadáním přesných rozměrů (například 1500×1250 mm). Addon otvor dynamicky vyřízne pomocí logiky Geometry Nodes (Mesh Boolean). Při jakékoliv změně rozměrů v panelu se otvor okamžitě zaktualizuje. Po dokončení celé dispozice uživatelka spustí finalizační nástroj, který aplikuje všechny modifikátory, zpracuje UV mapy a připraví čistou statickou síť (mesh) pro renderovací pipeline.

1.4.6 UC 2.3: Rychlá editace vlastností prvků přes kontextovou nabídku

Vizualizátorka potřebuje přiřadit různé materiály podlah jednotlivým místnostem, ideálně bez neustálého přepínání do postranních panelů. Klikne proto pravým tlačítkem myši na plochu místnosti ve viewportu. Vyvolaná kontextová nabídka zobrazí dostupné akce pro daný prvek. Uživatelka zvolí možnost „Změnit materiál podlahy“ a vybere odpovídající texturu. Stejným způsobem může místnost rovnou přejmenovat.

1.4.7 UC 3.1: Rychlý level blackout

Game designér aktivuje kreslicí nástroj a hrubě načrtne sérii navazujících místností. Soustředí se především na proporce a měřítko prostoru vůči hráčské postavě. Addon mezitím průběžně generuje stěny a detekuje vznikající místnosti. Uniformní výšku stěn pro celý půdorys designér následně nastaví v N-panelu.

1.4.8 UC 3.2: Finalizace a export herní úrovně

Na hotovém blackoutu přidá game designér dveřní otvory zadáním požadovaných parametrů v N-panelu u vybraných stěn. Zkontroluje podlahové plochy místností zobrazené v panelu a spustí finalizační nástroj. Ten aplikuje modifikátory Geometry Nodes, zkonvertuje atributy UV map a připraví statickou geometrii pro bezproblémový export do herního enginu ve formátu FBX nebo glTF.

1.4.9 UC 3.3: Interaktivní úprava rozložení místností

Game designér po herním testování (playtestingu) zjistí, že chodba mezi dvěma arénami je pro pohyb hráče příliš úzká. Vybere proto uzel na okraji chodby a pomocí 3D manipulátoru (gizmo) bod plynule posune v rovině XY. Addon automaticky udržuje planaritu a v reálném čase přepočítává sousední místnosti. Výslednou šířku chodby designér ověří čistě vizuálně ve viewportu, aniž by musel zadávat přesné číselné hodnoty.

1.5 Analýza požadavků

Z definovaných person a scénářů použití vyplývá, že modul musí umožňovat interaktivní kreslení půdorysu, parametrickou úpravu stěn a otvorů, automatickou detekci místností i přípravu modelu pro další zpracování (rendering či nasazení v herním enginu). Ne všechny tyto funkce jsou však pro jednotlivé cílové skupiny stejně důležité. Tato sekce proto strukturovaně rozděluje zjištěné potřeby do sedmi funkčních a tří nefunkčních požadavků a ústí v prioritizační analýzu, která hodnotí váhu každého požadavku napříč cílovými skupinami.

1.5.1 Funkční požadavky

Následujících sedm funkčních požadavků pokrývá celý zamýšlený rozsah modulu – od interaktivního kreslení přes parametrickou správu prvků až po finalizaci modelu a integraci pomocných grafických vrstev.

1.5.1.1 FP1 — Interaktivní tvorba místností a kreslení (Pencil Tool)

Nástroj pro kreslení představuje primární vstup addonu. Umožňuje uživateli definovat půdorys klikáním bodů přímo ve 3D scéně. Jádrem je modální operátor, který po dobu kreslení přebírá veškerou interakci s myší a klávesnicí a průběžně generuje stěny.

Nezbytným minimem pro realizaci je kreslení půdorysu v pohledu shora a spolehlivá správa uživatelských vstupů modálním operátorem. Důležitým rozšířením je automatické přichytávání (snapping) k osám XY odvozené od vzdálenosti kurzoru k existujícím bodům či osám. Jako volitelné vylepšení se nabízí průběžné vykreslování náhledu budoucí stěny před jejím potvrzením – systém by neustále sledoval pozici kurzoru, kreslil vodící linku a čekal na potvrzení uživatelem.

1.5.1.2 FP2 — Generování a úprava parametrických objektů

Tento požadavek definuje parametrické chování všech prvků půdorysu, tedy stěn i otvorů. Každý objekt si uchovává své parametry (délku, výšku, tloušťku, pozici v prostoru). Při jejich změně se automaticky přepočítá geometrie prvku a dynamicky se zaktualizuje poloha případných navázaných otvorů.

Základní implementace vyžaduje dynamickou reprezentaci stěn formou parametrického systému (nikoliv jako statickou síť), okamžitou aktualizaci geometrie při úpravě hodnot, zachování relativní pozice otvorů vůči stěně pomocí pevných datových vazeb a inteligentní generování ořezů (Boolean operací) přímo prostřednictvím Geometry Nodes.

1.5.1.3 FP3 — Správa prostoru a metadat

Správce prostoru tvoří sémantickou vrstvu nad obyčejnou 3D geometrií. Modul musí umět automaticky detekovat uzavřené cykly stěn, rozpoznat je jako samostatné místnosti a vypočítat jejich plochu. Jako volitelné rozšíření je koncipována hierarchizace prostorů – organizace místností a celých podlaží do přehledných kolekcí, které umožní například hromadné přepínání viditelnosti v projektu.

1.5.1.4 FP4 — Finalizační nástroj

Finalizační nástroj uzavírá nedestruktivní fázi návrhu. Po dokončení úprav převede parametrický systém na čistou, statickou 3D geometrii, která je připravená pro UV mapování, export do herního enginu nebo nasazení v renderovací pipeline. Hlavním požadavkem je trvalá aplikace všech procedurálních generátorů a modifikátorů u vybraných objektů scény.

1.5.1.5 FP5 — Kontextová nabídka

Kontextová nabídka zpřístupňuje akce vázané na konkrétní prvek pomocí plovcí uživatelské nabídky zobrazené přímo u kurzoru. Addon by měl využívat metodu vržení paprsku (raycast) k identifikaci cílového objektu a přes moduly `GPU` nebo `BLF` vykreslovat vlastní rozhraní překrývající 3D viewport.

1.5.1.6 FP6 — Interaktivní 3D manipulátory

Interaktivní 3D manipulátory (gizma) nabízejí alternativu k ručnímu vypisování parametrů. Umožňují geometrickou manipulaci přímo v prostoru: uživatel uchopí barevné grafické táhlo u daného prvku a tažením myši plynule mění jeho rozměry nebo výšku. Implementace by měla využít nativní rozhraní `bpy.types.Gizmo` a `GizmoGroup`.

1.5.1.7 FP7 — Automatické kótování

Kótovací vrstva průběžně zobrazuje délky stěn a plochy místností jako dynamický text přímo ve viewportu, takže uživatel nemusí zjišťovat rozměry v postranních panelech. Texty generované modulem `BLF` přes překreslovací smyčku (draw handler) se musejí aktualizovat v reálném čase při každé změně dispozice.

1.5.2 Nefunkční požadavky

1.5.2.1 NP1 — Architektura a technologie

Výpočetní jádro modulu je postaveno na Geometry Nodes: logika generování a tvarování geometrie probíhá uvnitř uzlových stromů, zatímco Python plní roli správce, který tyto stromy propojuje a dynamicky upravuje jejich vstupy. Toto striktní oddělení vizuální a aplikační logiky je klíčovým architektonickým principem. Z hlediska distribuce nesmí modul vyžadovat ruční doinstalaci externích knihoven. Cílová platforma (Blender 4.2+) umožňuje definovat závislosti přímo v konfiguračním souboru `blender_manifest.toml`. Externí knihovny, jako například `NetworkX` využívaná pro grafové výpočty

(detekce cyklů, planární embedding), tak budou zabaleny jako standardní Wheel soubory (.whl) a nainstalují se automaticky při aktivaci addonu.

1.5.2.2 NP2 — Výkon a nedestructivní přístup

Systém musí reagovat naprosto plynule: uživatel si musí zachovat možnost interaktivní úpravy i při komplexnějších změnách půdorysu a souběžném překreslování geometrie. Hlavním architektonickým důrazem je zde minimalizace výpočetní náročnosti, optimalizace přepočtů parametrů a důsledné respektování grafu závislostí v Blenderu (DepsGraph), aby nedocházelo k neefektivním cyklickým přepočtům celé 3D scény.

1.5.2.3 NP3 — Použitelnost a UX (Uživatelská zkušenost)

Uživatelské rozhraní modulu musí působit jako nativní součást Blenderu. Toho bude dosaženo konzistentním využíváním zabudovaných **UI** komponent (např. `UILayout.row`) a logickým seskupováním nástrojů do přehledných záložek s vysvětlujícími popisky (tooltips). Důraz je kladen na ošetření chyb a srozumitelnou zpětnou vazbu při pokusu o neplatnou operaci – např. při snaze vložit velké okno do příliš krátké stěny.

1.5.3 Prioritizace požadavků

Následující tabulka mapuje každý funkční balíček na scénáře použití, které ho motivují.

■ **Tabulka 2** Mapování funkčních balíčků na scénáře použití (● = relevantní)

Po- ža- da- vek	UC 1.1	UC 1.2	UC 2.1	UC 2.2	UC 3.1	UC 3.2	UC 1.3	UC 2.3	UC 3.3
FP1		●	●		●				
FP2	●	●	●	●	●	●			
FP3	●	●	●		●	●			
FP4				●		●			
FP5								●	
FP6									●
FP7							●		

Každý požadavek je hodnocen zvlášť každou cílovou skupinou (Vysoká / Střední / Nízká / Irelevantní) a výsledná priorita se určuje jako vážený průměr

s koeficienty: architekti $\times 3$, vizualizátoři $\times 2$, game designéři $\times 1$. Výsledný průměr $\geq 2,50$ odpovídá prioritě Vysoká (must-have), rozmezí 2,50–1,75 prioritě Střední (should-have) a zbytek je hodnocen jako Nízká (nice-to-have) priorita.

Tabulka 3 Vážená prioritizace požadavků podle cílových skupin

Poža- davek	Architekti	Vizualizátoři	Game Des.	Průměr	Priorita
FP1 — Kresle- ní	Vysoká	Vysoká	Vysoká	3,00	Vysoká
FP2 — Para- metry	Vysoká	Vysoká	Vysoká	3,00	Vysoká
NP1 — Archi- tektura	Vysoká	Vysoká	Vysoká	3,00	Vysoká
NP2 — Výkon	Vysoká	Vysoká	Vysoká	3,00	Vysoká
NP3 — UX	Vysoká	Vysoká	Vysoká	3,00	Vysoká
FP6 — Mani- puláto- ry	Střední	Střední	Nízká	1,83	Střední
FP3 — Prosto- ry	Vysoká	Nízká	Irelevantní	1,83	Střední
FP7 — Kóto- vání	Vysoká	Nízká	Irelevantní	1,83	Střední
FP4 — Finali- zace	Nízká	Střední	Vysoká	1,67	Nízká
FP5 — Kon- text. nabíd- ka	Střední	Nízká	Nízká	1,50	Nízká

Tabulka nám v další kapitole návrhu poslouží jako výchozí bod pro určení definice minimálního funkčního produktu a jeho hranic.

1.6 Technická analýza

Funkční požadavky říkají *co* má addon umět; technická analýza odpovídá na otázku *jak* — které části Blender API to umožňují, jaké jsou jejich limity a kde hrozí designové nedostatky, které by ovlivnily celou architekturu. Klíčové otázky jsou, jak zachytit kreslení půdorysu v reálném čase bez ztráty výkonu, jak reprezentovat půdorys jako responsivní datový model schopný detekovat místnosti a reagovat na každou změnu stěny, a jak parametrické objekty převést do statické geometrie připravené pro export.

1.6.1 Architektura Blenderu

Blender je modulární systém postavený na unikátním způsobu správy dat — dualitě systému `DNA` a `RNA`. `DNA` (Blender's Data Architecture) definuje struktury C pro veškerá interní data scény, zatímco `RNA` (Runtime Notification Architecture) poskytuje reflexivní API, přes které Python přistupuje k těmto strukturám a reaguje na jejich změny [11].

Blender využívá kombinaci vzoru `MVC` (Model-View-Controller), která umožňuje oddělit uživatelské rozhraní (View) od vnitřní logiky (Model) a zpracování vstupů (Controller). Addony mohou definovat vlastní výpočty například v Geometry Nodes (Model), zatímco Blender se stará o jejich vykreslení do viewportu a zachytávání událostí myši přes Python API.

1.6.2 Interaktivní kreslení a interakce ve viewportu

Plynulé kreslení stěny — sledování kurzoru, průběžná aktualizace náhledové linky a potvrzení kliknutím — vyžaduje zpracování každého z těchto kroků v rámci jednoho snímku. Tato sekce vysvětluje, jak Blender takovou interakci umožňuje přes modální operátory a stavový automat, a kde leží výkonnostní limit Pythonu, který je nutné obejít delegováním práce na C++ jádro.

1.6.2.1 Modální operátory

Interaktivní kreslení ve viewportu stojí na modálních operátorech — podtřídách `bpy.types.Operator` [12], které po spuštění zůstávají aktivní a naslouchají událostem myši a klávesnice. Na rozdíl od standardních operátorů, které vykonávají jednorázovou funkci a okamžitě skončí, modální operátor

kontinuálně naslouchá událostem generovaným uživatelem nebo systémem. Je nezbytný pro plynulé kreslení, kdy systém průběžně sleduje polohu kurzoru a dynamicky na ni reaguje.

Inicializace operátoru začíná metodou `invoke()`, která připraví počáteční stav a registruje operátor do modálního handleru správce oken pomocí `context.window_manager.modal_handler_add(self)`. Od tohoto momentu je každá událost ve viewportu předávána metodě `modal()`. Návrátové hodnoty `modal()` určují, jak Blender s událostí dále naloží:

■ **Tabulka 4** Návrátové hodnoty modálního operátoru

Hodnota	Funkční dopad	Architektonický význam
<code>RUNNING_MODAL</code>	Operátor pokračuje v běhu	Plynulé tažení stěny nebo změna rozměru
<code>PASS_THROUGH</code>	Operátor běží, událost předána dál	Zoomování a rotace pohledu během kreslení
<code>FINISHED</code>	Operátor končí, generuje Undo krok	Finalizace a uložení stěny do databáze
<code>CANCELLED</code>	Operátor končí bez uložení	Přerušování akce klávesou ESC

Metoda `modal()` funguje na principu stavového automatu, který umožňuje operátoru měnit chování v závislosti na fázi interakce. Pro kreslení půdorysu jsou typické stavy: **START** (čekání na první kliknutí), **DRAWING** (průběžné přepočítávání délky a úhlu stěny a vykreslování náhledové linky přes GPU modul), **EXTRUDING** (definování tloušťky nebo výšky) a **FINISHING** (zápis geometrie do scény a čištění draw handlerů).

Stavový automat přináší tři klíčové výhody: řízení složitosti při implementaci více krokových nástrojů, možnost kontextového snappingu (v různých stavech jsou aktivní různé typy přichytávání) a optimalizaci výkonu — složité operace se spouštějí pouze při přechodu mezi stavy, zatímco při pohybu myši se aktualizuje pouze drobná vizualizace.

1.6.2.2 Limity výkonu jazyka Python v programu Blender

Python je v programu Blender interpretovaným jazykem, proto je potřeba náročné operace delegovat na stranu programu Blender nebo GPU využívající C++. Zkušenosti z vývoje komplexních generativních nástrojů ukazují, že čistý Python je při hromadném zpracování dat řádově pomalejší. V kontextu architektonického kreslení jsou hlavními limitujícími faktory iterace přes mesh data pomocí `for` smyčky, rostoucí počet unikátních objektů ve scéně a časté aktualizace `DepsGraphu`.

K překonání těchto limitů se v profesionálních addonech využívají tři techniky. Metody `foreach_set` a `foreach_get` umožňují přenášet celá pole dat mezi jazykem Python a interními C++ strukturami v jedné operaci namísto nastavování každého vrcholu zvlášť. Delegování na modifikátory spočívá v tom, že Python vytvoří pouze základní čárový model a na něj aplikuje modifikátory jako Solidify nebo Bevel — ty jsou implementovány v C++, plně využívají multithreading a jsou optimalizovány pro real-time aktualizaci. Geometry Nodes jako výpočetní backend pak umožňují jazyku Python pouze manipulovat se vstupními hodnotami uzlového stromu, zatímco veškerý výpočet geometrie probíhá v nativním kódu programu Blender.

1.6.3 Reprezentace geometrie

Přesná tloušťka v ostrých rozích, interaktivní odezva na změnu parametru a čistá topologie vhodná pro UV mapování jsou tři protichůdné nároky, které ne každý přístup k reprezentaci geometrie splňuje najednou. Tato sekce analyzuje tři dostupné možnosti: imperativní BMesh, který nabízí maximální topologickou kontrolu za cenu výkonu; deklarativní Geometry Nodes, které výpočet přesouvají do nativního C++ jádra programu Blender; a hybridní přístup kombinující přesný výpočet geometrie v Pythonu s GN jako vykreslovacím backendem.

Geometrie (poloha prvků v prostoru) a topologie (vzájemné vztahy a propojení) tvoří základní dualitu jakékoli 3D struktury. V programu Blender je základní jednotkou mesh složená z vrcholů, hran a ploch.

1.6.3.1 Datová struktura BMesh

BMesh je interní datová struktura programu Blender, která na rozdíl od tradičních struktur založených na trojúhelnících podporuje n-gony (polygony s více než čtyřmi vrcholy). Využívá systém podobný half-edge datovým strukturám, kde jsou vztahy mezi plochami a hranami uloženy tak, aby umožňovaly rychlou navigaci po povrchu sítě.

Z pohledu parametrického modelování nabízí BMesh skrze Python API (modul `bmesh`) nízkourovňový přístup k topologii — možnost dotazovat se, které hrany jsou spojeny s daným vrcholem, a tím provádět operace jako `dissolve` bez poškození okolní topologie. Algoritmus pro generování stěn obvykle začíná načtením 2D hran, identifikuje uzavřené smyčky jako obrysy místností a operací tloušťky (`offset`) — například přes `bmesh.ops.bevel` nebo posunem vrcholů podél normál hran — vytváří 3D stěny. Tento proces je v Pythonu relativně pomalý a neumožňuje plynulou interaktivní odezvu, zejména při průběžné validaci integrity sítě. Na druhou stranu ale poskytuje absolutní kontrolu nad datovou strukturou a zaručuje bezchybnou, čistou topologii.

1.6.3.2 Geometry Nodes

Geometry Nodes ([GN](#)) zastupují deklarativní, paralelní přístup: uživatel definuje systém pravidel aplikovaných na celou geometrii současně. Data jsou reprezentována jako pole atributů vázaných na různé domény (vrcholy, hrany, plochy, instance), přičemž výpočet probíhá v nativním kódu C++ s plným multithreadingem.

Pro generování stěn se v GN nejčastěji používá uzel `Curve to Mesh`. Klíčovou výzvou je Miter Joint problém — standardní vytažení profilu podél křivky vede ke ztenčení stěny v ostrých rozích. Řešením je matematická korekce měřítka profilu v každém bodě křivky pomocí faktoru $f = \frac{1}{\sin(\frac{\theta}{2})}$, kde θ je úhel mezi sousedními segmenty stěny. Tento výpočet se v GN realizuje pomocí vektorové matematiky (skalární součin pro výpočet úhlu) a ač je komplexnější na přípravu, umožňuje dynamicky měnit tloušťku stěn pouhým posunutím bodu v 2D půdorysu. Zásadní slabinou GN je ovšem výsledná topologie. Zvláště po aplikaci booleovských operací (např. pro otvory oken a dveří) GN často generují nepředvídatelnou, triangulovanou nebo nevhodnou n-gonovou síť, která silně komplikuje čisté UV mapování a další manuální úpravy.

1.6.3.3 Hybridní přístup: Python quad-polygon a GN extrude

Třetí možností je kombinace obou předchozích přístupů, kde Python řeší geometricky náročné výpočty a Geometry Nodes (GN) fungují primárně jako vykreslovací backend. Důvodem je, že čistě programový ani čistě uzlový přístup nedokážou současně zajistit všechny tři úvodní požadavky: přesné napojení stěn, interaktivní odezvu i čistou topologii — každý z nich má v určitém ohledu slabinu.

Zvláštní pozornost je věnována rohům a křížení stěn pod různými úhly, kde by jednoduchý kolmý řez vytvářel mezery nebo nechtěné překryvy. Tento problém je řešen algoritmicky na straně Pythonu. Systém analyzuje všechny stěny v daném spoji, seřadí je podle úhlu odchozího směru a následně vypočítá přesné průsečíky jejich hran. Pro každou stěnu tak vznikne přesný 2D půdorys, který plně respektuje její osu a tloušťku. U složitějších spojů, jako jsou křížení ve tvaru T nebo X, algoritmus navíc generuje speciální výplňovou geometrii (*junction fill*), která spoj plynule uzavře a zabráni vzniku vizuálních děr v horní ploše křížení.

Vypočítané půdorysy jsou následně zapsány do základní sítě modelu společně s potřebnými metadaty, jako je cílová výška stěn. Role stromu Geometry Nodes je díky tomu zredukována na minimalistickou a stabilní sadu operací: vyfiltrování příslušného půdorysu, jeho vytažení do 3D prostoru na základě

předaných parametrů a následné vyříznutí otvorů pro architektonické prvky pomocí booleovských operací.

Zásadní výhodou tohoto řešení je, že úspěšně uzavírá pomyslný trojúhelník nároků definovaný v úvodu. Generování 2D půdorysů a spojů čistě v Pythonu zajišťuje naprostou přesnost tloušťky i čistou quad topologii, přičemž oddělenou logiku lze spolehlivě ověřovat pomocí standardních automatizovaných testů. Následné delegování 3D extruze na C++ jádro GN zase poskytuje potřebný výkon pro rychlou interaktivní odezvu při úpravách. Nevýhodou je naopak o něco náročnější implementace synchronizační vrstvy. Přenos vypočítaných vlastností z Pythonu do Geometry Nodes vyžaduje pečlivou správu dat a obcházení určitých limitací při zápisu interních atributů sítě. Celkové srovnání analyzovaných přístupů napříč klíčovými technickými parametry shrnuje [Tabulka 5](#).

■ **Tabulka 5** Srovnání přístupů k reprezentaci geometrie stěn

Charakteristika	BMesh	Geometry Nodes	Hybridní
Způsob práce	Iterativní / Imperativní	Paralelní / Deklarativní	Python geometrie + GN render
Výkon	Omezený interpretací Pythonu	Vysoce optimalizované C++	Python sync odložen na konec seance
Topologická flexibilita	Absolutní	Omezená na definované uzly	Plná (Python vrstva)
Vizuální odezva	Po spuštění skriptu	Real-time ve viewportu	GPU preview per-click; full sync na konci
Přesnost spojů	Přesné, výpočetně drahé	Vyžaduje manuální korekci	Přesné (angular sort)
Multithreading	Ne (omezení GIL)	Ano (nativní)	GN část ano; Python část ne
Testovatelnost	Omezená (závislost na bpy)	Obtížná	Plná (čistý Python, bez bpy)
Implementační složitost	Střední	Nízká	Vysoká (sync vrstva)

1.6.4 Datový model

Samotná 3D síť (mesh) sice přesně zachycuje tvar a polohu stěn v prostoru, ale nenese žádné sémantické informace o prostorech samotných — nemá jak zjistit, zda spolu dvě místnosti sousedí, jaký mají účel nebo jaké jsou jejich parametry. Parametrický modul, jehož úkolem je automaticky detekovat místnosti, počítat jejich podlahové plochy a dynamicky reagovat na každou topologickou změnu, proto nevyhnutelně vyžaduje dedikovanou datovou vrstvu. Tato sekce analyzuje možné přístupy k její reprezentaci.

1.6.4.1 Reprezentace bez samostatné datové vrstvy

Nejjednodušší možný přístup vůbec nevytváří oddělenou datovou strukturu: půdorys je uložen výhradně v geometrii Blenderu a veškerá sémantická data jsou vázána přímo na konkrétní prvky sítě pomocí uživatelských vlastností (Custom Properties) nebo pojmenovaných atributů (Named Attributes). V takovém modelu představují hrany sítě stěny a jednotlivé polygony (faces) tvoří místnosti.

Hlavní výhodou je jednoduchost — řešení nevyžaduje žádné externí knihovny a nabízí přímou kompatibilitu s modifikátory Geometry Nodes, které umějí pojmenované atributy číst v reálném čase. Zásadní nevýhodou je však velmi nízká efektivita dotazování. Hledání odpovědí na otázky typu „sousedí místnost A s místností B?“ nebo „kolik místností je dostupných z hlavní haly?“ vyžaduje iterativní procházení celé topologie sítě s časovou složitostí $O(E)$ pro každý jednotlivý dotaz.

1.6.4.2 Lineární datové struktury

Druhý přístup udržuje aplikační stav v Pythonu pomocí plochých seznamů nebo slovníků. Eviduje uzly (styky stěn), stěny a místnosti jako samostatné objekty. Každý prvek má přiřazen jednoznačný identifikátor (ID) a informace o sousednosti je uložena přímo v záznamu daného prvku jako seznam ID jeho sousedů.

Tato varianta je plně implementovatelná pomocí standardních knihoven Pythonu a lze ji snadno testovat i nezávisle na programu Blender. Jejím hlavním limitem je však rychlé snížení výkonu při složitějších topologických operacích. Například automatická detekce nově vzniklých místností (uzavřených cyklů) by vyžadovala implementaci vlastních algoritmů pro prohledávání grafu a výpočetní náročnost při opakovaném přepočítávání sousednosti by expocencionálně rostla s každým novým prvkem.

1.6.4.3 Grafová reprezentace

Z matematického hlediska je nejpřirozenějším modelem půdorysu neorientovaný planární graf [13], kde uzly (V) odpovídají stykům stěn a hrany (E) představují osy samotných stěn. Tento formát otevírá cestu k využití standardních, vysoce optimalizovaných grafových algoritmů: od detekce minimálních cyklů (místností), přes ověřování planarity, až po výpočty dostupnosti. Pro Python navíc existuje robustní knihovna NetworkX [14], která všechny tyto operace nativně implementuje a umožňuje jejich testování zcela nezávisle na Blenderu.

Hlavním architektonickým rozhodnutím při použití grafové reprezentace je hloubka struktury modelu: tedy zda zachovat jeden sjednocený graf kombinující topologii se sémantikou (uzly nesou jak geometrické souřadnice, tak metadata místnosti), nebo striktně oddělit topologický skelet (uzly = spoje stěn, hrany = osy stěn) od sémantického grafu (uzly = místnosti, hrany = sousedství). Sjednocený graf je jednodušší na průběžnou správu. Oddělené vrstvy sice vyžadují synchronizaci, ale mnohem lépe izolují zodpovědnosti jednotlivých domén. Rozdíl ve výkonu je zde markantní: zatímco v čistě topologickém grafu má zjištění sousednosti dvou místností složitost $O(E)$, v explicitním sémantickém grafu jde o triviální dotaz se složitostí $O(1)$.

S grafovou reprezentací úzce souvisí i strategie automatické detekce místností při uzavírání stěnových cyklů, jíž se detailně věnuje následující podkapitola.

1.6.4.4 Způsoby detekce místností

Pro detekci místností existují dva přístupy. *Eager* přístup ihned při vzniku stěny vytváří dočasný objekt místnosti, který je při uzavření cyklu validován. Toto vede k řadě problémů: nedefinovanému stavu dočasného objektu, konfliktu při slučování po uzavření cyklu (systém musí vybrat, který dočasný objekt zachovat), problémům se simultánním uzavřením více cyklů a složité vlastní správou Undo historie.

Lazy přístup naproti tomu vytváří místnost výhradně tehdy, kdy je detekován uzavřený cyklus v strukturálním grafu. Invariant `Room` ↔ `minimální uzavřený cyklus` platí vždy a plně — neexistuje žádný stav „rozpracované místnosti“. NetworkX vrátí seznam všech nových minimálních cyklů najednou, pro každý vznikne jeden `Room` node bez spojovací logiky. Undo je přirozené: odebrání uzavírající stěny znamená zánik cyklu a zánik `Room` nodu. *Lazy* detekce zajišťuje determinismus — stejná topologie spojů a stěn vždy produkuje stejnou sémantiku místností bez závislosti na pořadí editací.

1.6.5 Tvorba otvorů pro okna a dveře

Nedestruktivní přístup k otvorům předpokládá, že pohyb stěny automaticky přesune i zabudované okno, změna tloušťky ho přizpůsobí a aktualizace parametrů otvor okamžitě regeneruje — bez jakéhokoli ručního zásahu. Splnit ji lze více způsoby, přičemž každý nabízí jiný kompromis mezi výkonem, numerickou stabilitou a topologickou čistotou výsledné geometrie. Tato sekce porovnává následující tři hlavní přístupy jak zmíněnou podmínku splnit:

Boolean operace spravované přes Python API využívají standardní modifikátorový stack, kde Python dynamicky vytváří cutter objekty a přiřazuje je ke stěně. Hlavní nevýhodou je vysoká režie při správě stacku s desítkami modifikátorů a numerická nestabilita — pokud souřadnice po transformaci nejsou binárně identické (kvůli zaokrouhlení floatů), Exact solver může selhat při detekci společných ploch.

Mesh Boolean v Geometry Nodes nahrazuje objektový stack jedním uzlovým stromem, kde operace probíhají nad toky geometrických dat. Na rozdíl od modifikátorů, které pracují v párech, uzel Mesh Boolean v GN dokáže zpracovat celé kolekce instancí oken jako jeden sloučený vstup — to dramaticky snižuje počet reevaluací. Problémem je, že solver považuje vše zapojené do vstupu za jediné těleso; pokud se dvě stěny překrývají, může dojít ke vzniku dutých výsledků.

Metody bez Boolean operací se vyhýbají výpočtům průsečíků a otvory vkládají přímo do procesu generování topologie. Curve Trimming ořízne vodící křivku před vytažením do 3D, čímž vzniknou fyzické mezery s čistou topologií — tato metoda však vyžaduje reprezentaci stěn jako Blender Curve objektů a není kompatibilní s architekturou pracující s base meshem a Named Attributes. Modulární instancování chápe stěnu jako pole buněk s plnými nebo prázdnými moduly. Vertex Group Topology označí specifické vrcholy atributem `is_window` a GN je posunou aby vytvořily pravoúhlý otvor.

Následující tabulka zobrazuje hlavní rozdíly mezi zmíněnými přístupy:

■ **Tabulka 6** Srovnání metod pro tvorbu otvorů ve stěnách

Parametr	API Modifiká-tory	GN Mesh Boo-lean	Curve Trim-ming
Výpočetní složi-tost	$O(n \times m)$	$O(m \log m)$	$O(n)$
Numerická stabi-lita	Nízká (float drift)	Střední	Absolutní
Topologický vý-stup	Artefakty, n-go-ny	n-gony, nutný cleanup	Perfektní quads
Flexibilita	Vysoká	Vysoká	Omezená

1.6.6 Ukládání dat a správa metadat

Parametrický addon pracuje s daty na dvou odlišných úrovních: s geometrií stěn, která musí přežít každý zpětný (Undo) krok, i s metadaty místností — jejich názvem, typem a plochou — která musí zůstat konzistentní při sdílení souborů nebo instancování objektů. Tato sekce analyzuje, které mechanismy Blender API pro tato data nabízí, na které úrovni hierarchie programu Blender je přirozeně ukládat a jak zajistit, aby grafové struktury v paměti přežily uložení a opětovné otevření `.blend` souboru.

1.6.6.1 Systémy pro správu uživatelských parametrů

Blender nabízí dva hlavní mechanismy. **Vlastnosti ID** (Custom Properties) jsou flexibilní mechanismus pro připojování libovolných dat k jakémukoliv datovému bloku — ukládají celá čísla, desetinná čísla, řetězce a pole. Pro komplexní architektonické systémy mají nedostatky: absenci striktní typové kontroly a omezené možnosti definice logiky při změně hodnoty. **Modul `bpy.props`** umožňuje definovat vlastnosti registrované přímo do systému RNA s plnou podporou zpětných volání (`update`, `get`, `set`), klíčových pro reaktivní architektonické prvky.

Pro správu informací o prostorech (název, plocha, typ místnosti, výška) je optimálním vzorem `bpy.types.PropertyGroup`, který seskupuje související parametry do logického celku připojeného k datovému bloku pomocí `PointerProperty` (vazba 1:1) nebo `CollectionProperty` (vazba 1:N).

1.6.6.2 Vazby dat na úrovně hierarchie Blenderu

Rozhodnutí, na jakou úroveň hierarchie Blenderu budou metadata uložena, má hluboké důsledky pro stabilitu addonu a chování při Undo/Redo.

Úroveň **Scéna** (`bpy.types.Scene`) je vhodná pro globální parametry projektu. Data specifická pro jednotlivé místnosti jsou na této úrovni nevhodná: smazání objektu ve viewportu nezpůsobí automatické smazání metadat v kolekci, což vede k hromadění dat. Úroveň **Objekt** (`bpy.types.Object`) nese informace o transformaci a viditelnosti. Komplikace nastávají při instancování: vytvoření instance přes Alt+D vytvoří dva objekty sdílející stejná geometrická data, ale s unikátními daty na úrovni Object — změna rozměru jednoho okna by se neprojevila u ostatních instancí. Úroveň **Geometrie** (`bpy.types.Mesh`) je nejvhodnějším přístupem pro architektonické prvky: mesh reprezentuje „definici typu“, prvku a všechny sdílené instance mají identické parametry, v souladu s principy BIM.

1.6.6.3 Perzistence grafových dat

Blender při uložení `.blend` souboru automaticky ukládá mesh geometrii a Custom Properties objektů — Python objekty v paměti (např. NetworkX grafy) nikoli. Po zavření a opětovném otevření souboru jsou grafy ztraceny, pokud nejsou zachovány.

■ **Tabulka 7** Přístupy k perzistenci grafových dat v Blenderu

Přístup	Princip	Hlavní nevýhoda
JSON v Custom Property	Serializovat grafy do JSON stringu	Redundance; nutno verzovat schéma
Pickle v Custom Property	Serializovat Python objekty do bajtů	Bezpečnostní riziko; citlivost na verze
Rekonstrukce z meshe	Po načtení přebudovat grafy z uložené topologie	Mesh musí být jediným zdrojem pravdy

1.6.6.4 Perzistence globálních nastavení addonu

Addon spravuje dvě kategorie nastavení. **Projektová data** (výchozí tloušťka stěny, výška, systém jednotek) přímo ovlivňují geometrii konkrétního půdorysu. **Nastavení chování addonu** (výkon, cesty k souborům, UI preference) jsou naopak nezávislá na projektu.

Pro každou kategorii nabízí Blender jiný mechanismus: `bpy.types.AddonPreferences` ukládá data globálně do `userpref.blend` (platí napříč projekty), zatímco `bpy.types.PropertyGroup` na Scene ukládá data per-projekt do aktuálního `.blend` souboru. Analýza existujících addonů ukazuje, že oba hlavní architektonické addony (Archimesh, Archipack) projektová data do `AddonPreferences` neukládají — ten rezervují výhradně pro chování

addonu. Pro FloorPlanMaster je proto optimálním řešením **Scene PropertyGroup** se zabezpečenými výchozími hodnotami: projektové hodnoty jsou součástí `.blend` souboru, jsou tedy přenositelné a verzovatelné s projektem. `AddonPreferences` se použijí výhradně pro nastavení nesouvisející s konkrétním projektem.

1.6.7 Uživatelské rozhraní

Architekt, který musí odtrhnout pohled od půdorysu, aby v postranním panelu dohledal tlačítko, ztrácí soustředění právě ve chvíli, kdy je nejcennější. Dobrý UI addon proto nesmí nutit uživatele hledat — parametry stěny se musí zobrazit samy při jejím výběru, délky stěn musí být čitelné přímo ve viewportu a editace hodnotou i tažením myši musí být dostupné ze stejného místa. Tato sekce popisuje, jaké mechanismy Blender API pro takové rozhraní nabízí a jaké UI vzory se v existujících architektonických nástrojích opakují a proč.

UI systém Blenderu je postaven na principu Immediate Mode rendering — rozhraní se kompletně překresluje při každém snímku nebo změně. Logika určující, co se má zobrazit, musí být extrémně rychlá; výhodou je flexibilita — rozhraní vždy dokonale odráží aktuální stav bez synchronizace. Prostor obrazovky je hierarchicky členěn: Screen (celé hlavní okno), Area (obdélníkové oblasti) a Region (specifické části každé oblasti, v 3D Viewportu to jsou hlavní 3D zobrazení, Sidebar, Header a Toolbar). K propojení UI s daty scény slouží RNA systém: data v rozhraní jsou pouze vizuálním ukazatelem do datových struktur C++ jádra a uživatelská změna hodnoty se okamžitě propíše přes RNA přímo do databáze scény.

1.6.7.1 GPU modul a draw handlers

Modul `gpu` slouží jako abstrakční vrstva nad nízkoúrovňovými grafickými API (OpenGL, Metal, Vulkan) a je pro architektonický addon klíčový: umožňuje vykreslovat vodící linky, kóty a náhledy stěn přímo na GPU bez nutnosti vytvářet geometrii v databázi Blenderu. Draw handlers se registrují k `bpy.types.SpaceView3D` metodou `draw_handler_add`. Existují dva hlavní režimy: `POST_VIEW` pracuje v souřadném systému 3D scény (ideální pro vodící linky) a `POST_PIXEL` pracuje v souřadnicích obrazovky a je nezbytný pro textové popisky a kóty, které mají zůstat čitelné nezávisle na míře přiblížení. Každý přidaný handler musí být při ukončení operátoru odstraněn pomocí `draw_handler_remove()`.

Pro kótovací popisky lze použít modul BLF (Blender Font Library), který vykresluje text přímo do viewportu s možností kontroly nad pozicí a vzhledem. Pro kótovací overlay je rozhodující čitelnost nezávislá na úhlu pohledu —

GPU POST_PIXEL overlay tuto podmínku splňuje; alternativní přístup s GN String to Curves generuje 3D mesh textu, který je při šikmém pohledu hůře čitelný.

1.6.7.2 UI vzory v architektonických nástrojích

Analýza existujících řešení odhalila opakující se UI vzory, které cílová uživatelská skupina již ovládá. **Sidebar / Properties panel** — parametry vybraného objektu jsou trvale viditelné v postranním panelu; Archipack posouvá tento vzor dál automatickým zobrazením parametrů v N-panelu při výběru objektu bez nutnosti klikat na tlačítko. **Gizmo při výběru (Auto-manipulate on select)** — při výběru stěny se automaticky zobrazí táhla pro tloušťku a výšku, vhodné pro geometrické úpravy tažením myši. **HUD během modálního nástroje** — aktuální hodnoty a nápověda kláves zobrazeny přímo ve viewportu, nezbytné pro nástroje s více stavy. **Kontextová nabídka na RMB** — přístup k méně frekventovaným akcím (přejmenování, smazání, nastavení materiálu) je primární UI konvencí Blenderu; nabídka je závislá na vybraný prvek, čímž redukuje vizuální šum. **Pop-over dialog** — otevírá se u kurzoru, poskytuje prostor pro textový vstup nebo výběr z enumu a zavře se kliknutím mimo bez nutnosti potvrzení. **Jednoznakové zkratky** — odpovídající prvnímu písmenu akce nebo ustálené konvenci prostředí, snižují latenci opakovaných akcí bez přepnutí pohledu na toolbar.

1.6.8 Finalizační nástroj

Parametrický model funguje jako živý organismus plný vzájemných závislostí: Geometry Nodes v reálném čase přepočítávají tvary geometrie, pojmenované atributy uchovávají data o místnostech a modifikátory čekají na své konečné uplatnění. Renderovací a herní enginy však vyžadují přesný opak – statickou geometrii (mesh) s čistými UV kanály a bez zbytečných materiálů, která už není nijak svázána s původním procedurálním výpočtem. Tato sekce analyzuje, jak takovou konverzi bezpečně provést pomocí vyhodnoceného grafu závislostí (Evaluated Depsgraph), aby nedošlo ke ztrátě dat ani k nevratnému zničení původního modelu.

Nástroj pro finalizaci tvoří pomyslnou tečku za nedestruktivní fází návrhu. Jeho úkolem je převést procedurální geometrii do statické, topologicky čisté a plně optimalizované sítě. Technicky se tento proces opírá o rozdíl mezi původními daty (Original Data, tedy čistou sítí bez modifikátorů) a vyhodnoceným objektem (Evaluated Object), což je stav po aplikaci všech modifikátorů a uzlových stromů. Vzhledem k tomu, že Blender používá pro správu vztahů mezi objekty centrální graf závislostí (DepsGraph), je pro finalizaci klíčové získat právě jeho vyhodnocenou verzi, která reprezentuje finální stav scény.

Abychom získali čistou síť připravenou pro export, je nezbytné načíst vyhodnocený stav objektu pomocí metody `evaluated_get()` a z něj následně vygenerovat novou statickou geometrii voláním `bpy.data.meshes.new_from_object(obj_eval)`. Tento postup zaručuje naprostou nedestruktivnost – původní parametrický objekt zůstává nedotčen a uživatel se k němu může kdykoliv vrátit. Naopak běžný postup, který spoléhá na procházení seznamu `obj.modifiers` a postupné volání `modifier_apply()`, bývá vysoce rizikový. Každá úspěšná aplikace modifikátoru totiž změní indexy těch zbývajících. V praxi se proto osvědčují spolehlivější přístupy: buď iterace přes předem připravený statický seznam názvů modifikátorů, nebo použití cyklu `while` s integrovaným ošetřením chyb, který problematické položky jednoduše přeskočí.

Zvláštní pozornost vyžadují UV mapy a materiály. V prostředí Geometry Nodes jsou UV mapy ukládány pouze jako 2D vektory v rozích polygonů (v doméně Face Corner). Pokud by zůstaly v této podobě, exportéry do formátů FBX nebo glTF by je zcela ignorovaly. Po aplikaci procedurální geometrie je proto nezbytné tyto pojmenované atributy explicitně převést na standardní UV vrstvy. Další komplikace vzniká při slučování instancí – často totiž dochází ke kumulaci geometrií s odlišnými definicemi, čímž vznikají zbytečně duplicitní materiálové sloty. V herních enginech by tento stav vedl k nežádoucímu nárůstu vykreslovacích požadavků (draw calls). Závěrečná finalizace by proto měla zahrnovat důkladné čištění materiálů, při kterém nástroj zanalyzuje existující sloty, identifikuje duplikáty, pomocí modulu BMesh přemapuje indexy a následně odstraní veškeré prázdné a nadbytečné pozice.

Tato kapitola převádí FloorPlanMaster do realizovatelného technického návrhu: vymezuje přesné hranice **MVP**, aby bylo jasné, které části (kreslení stěn, detekce místností, parametrické otvory, finalizace meshe) tvoří jádro první verze a které prvky zůstávají odloženy. Následně definuje třívrstvou architekturu, v níž strukturální graf drží topologii stěn a junctions, graf místností odvozuje semantiku uzavřených cyklů a synchronizační vrstva zapisuje data do pojmenovaných atributů meshe pro Geometry Nodes. Návrh dále konkretizuje datové entity a jejich vazby (Junction, Wall, Room, Adjacency), způsob propagace změn od uživatelské akce po přegenerování geometrie, pravidla práce operátorů a podobu rozhraní ve viewportu i panelech. Závěr kapitoly ověřuje konzistenci návrhu vůči scénářům použití a připravuje podklad pro implementaci bez doplňování chybějících rozhodnutí.

2.1 Definice rozsahu MVP

Funkční požadavky FP1 až FP7 identifikované v analýze (**Tabulka 3**) pokrývají kompletní workflow od prvního načrtnutí dispozice až po finální export pro herní engine či renderovací pipeline. Celý tento rozsah nelze realizovat najednou — pokus o to by vedl k nekonzistentní a nedokončené implementaci. Tato sekce proto zavádí MoSCoW prioritizaci jako filtr: musí být zcela jasné, které dílčí prvky tvoří nezbytné jádro MVP, které jsou hodnotnou, ale volitelnou nadstavbou a které lze realizovat až v pozdějších iteracích. Toto rozlišení prostupuje celým návrhem a v každé z následujících sekcí je každá funkce explicitně označena svou prioritou.

MVP realizuje kompletní workflow jednoho podlaží: interaktivní kreslení stěn, automatickou detekci místností, parametrické otvory a finalizaci do statické geometrie. Must-have prvky tvoří minimální sadu, bez níž addon nesplňuje základní účel a nemůže být nasazen.

■ **Tabulka 8** MoSCoW prioritizace prvků FP1–FP7; must-have prvky tvoří jádro MVP

Požadavek	Must-have základ	Odložené prvky
FP1	Kreslení bodů klikáním v top view; modální operátor; pre-view stěny	Snap na osu, mřížku a úhel; pokročilý GPU overlay
FP2	Dynamické parametry stěny; update geometrie; vazba otvorů; GN Mesh Boolean	Napojení místnosti na existující půdorys
FP3	Automatická detekce uzavřených místností; zobrazení plochy	Hierarchizace místností
FP4	Finalizace — aplikace GN modifikátoru, konverze UV, konsolidace materiálů	—
FP5	—	Kontextová nabídka a raycast elementů (should-have)
FP6	—	Interaktivní 3D manipulátory (should-have)
FP7	—	Automatické kótování (should-have)

Záměrně vyloučeny z celého rozsahu návrhu jsou: více podlaží a hierarchie budov, import a export formátů DXF a IFC, generování střech a schodišť a napojení nové místnosti na existující geometrii při vkládání z parametrů. Tato omezení nejsou chybami návrhu — architektura je na tato rozšíření připravena, avšak jejich detailní specifikace je v této fázi záměrně odložena do navazujících iterací jako budoucí rozšiřující funkčnost.

2.2 Architektura systému

Analýza odhalila klíčový problém: Blender [1] jako obecný 3D nástroj zná geometrické primitivy — vrchol, hranu, plochu — ale nezná stěnu, junction ani místnost. Bez sémantické vrstvy by každý objekt v scéně byl pouhým seskupením polygonů bez doménového kontextu a jakákoli změna půdorysu by vyžadovala manuální přepočítání okolní geometrie. Architektura FloorPlan-Masteru tuto mezeru zaplňuje oddělením sémantické logiky v Pythonu od vizualizace v Blenderu: Python vlastní veškeré znalosti o stěnách, místnostech a jejich vztazích; Blender slouží výhradně jako zobrazovací engine. Analogický

přístup volí Revit [10], ArchiCAD a FreeCAD Arch — jsou postaveny na obecném geometrickém jádru, ale přidávají nad ním sémantickou vrstvu entit (stěna, místnost, podlaží) s vlastními omezeními a vztahy.

2.2.1 Proč oddělená sémantická vrstva

Oddělení sémantiky od Blender dat není pouze implementační preference, ale vědomé architektonické rozhodnutí. Jádro problému spočívá v tom, že požadované operace addonu (detekce cyklů místností, stabilní identita stěn při editaci, validace topologie, sousednosti mezi místnostmi) jsou primárně grafové a relační, zatímco Blender mesh je primárně geometrická reprezentace optimalizovaná pro zobrazení a modelovací operace. Pokud by sémantika vznikala jen „zpětným čtením“ z meshe, každá složitější editace by vyžadovala opakovanou rekonstrukci doménových vztahů z vrcholů, hran a ploch.

V prostředí Blenderu existují i alternativní cesty, jak sémantiku řešit:

Tabulka 9 Alternativy reprezentace sémantiky v Blender addonu a jejich trade-offy

Přístup	Hlavní výhody	Hlavní omezení
Bez samostatné sémantické vrstvy (mesh-first)	Jednodušší datový tok; jeden zdroj dat přímo v geometrii; přirozená vazba na Undo/Redo a uložení .blend	Složitá a výpočetně náročná rekonstrukce významu z topologie; obtížná stabilita identit při změnách; slabší testovatelnost mimo Blender
Oddělený Python model + synchronizace do meshe (zvolený)	Přirozené vyjádření grafových vztahů; čisté API modelu; jednotkové testy bez Blenderu; predikovatelné chování při evoluci funkcí	Nutnost navrhnout robustní synchronizaci a perzistenci; vyšší implementační složitost; riziko desynchronizace při chybách v bridge vrstvě

Zvolená architektura tedy nepředpokládá, že ostatní varianty jsou „špatné“; pouze přesouvá optimalizační cíl od krátkodobé jednoduchosti k dlouhodobé konzistenci doménového modelu. Nevýhody zvoleného směru (synchronizační režie, potřeba explicitní perzistence a rekonstrukce) jsou přijaty záměrně, protože umožňují stabilní rozvoj funkcí nad stejným modelem dat i v dalších iteracích (více podlaží, nové typy prvků, exportní adaptéry) bez nutnosti měnit základní princip architektury.

2.2.2 Třívrstvá hybridní architektura

Jádrem návrhu je architektura, která striktně odděluje výpočetní logiku od vizualizace. Celý systém je postaven na třech specializovaných vrstvách: první dvě jsou implementovány výhradně v jazyce Python a zajišťují topologii a sémantiku půdorysu, zatímco třetí vrstva slouží jako jednosměrný most do vykreslovacího jádra programu Blender.

2.2.2.1 Vrstva 1: Topologický základ půdorysu

Vrstva 1 (strukturální graf) tvoří primární strukturu dispozice. Definuje počáteční a koncové body stěn, jejich vzájemné návaznosti a hlavní parametry. Díky tomu systém neustále pracuje s jednoznačně určenou strukturou dispozice a dokáže zajistit, že nedochází k nekonzistencím v geometrickém uspořádání stěn. Detailní datový model první vrstvy je popsán v kapitole 3.3.

2.2.2.2 Vrstva 2: Sémantický graf místností

Vrstva 2 (sémantický graf místností) je přímo odvozena z první vrstvy. Jakmile stěny vytvoří uzavřený prostor, systém jej automaticky detekuje jako místnost a průběžně vypočítává její základní parametry, jako jsou celková plocha a obvod. Každá místnost disponuje trvalým identifikátorem a uživatelsky definovaným názvem, což umožňuje její spolehlivou identifikaci i během následných úprav dispozice. Sdílejí-li dvě místnosti společnou stěnu, systém tuto vazbu eviduje jako sousednost, čímž jasně mapuje vzájemné propojení jednotlivých prostorů.

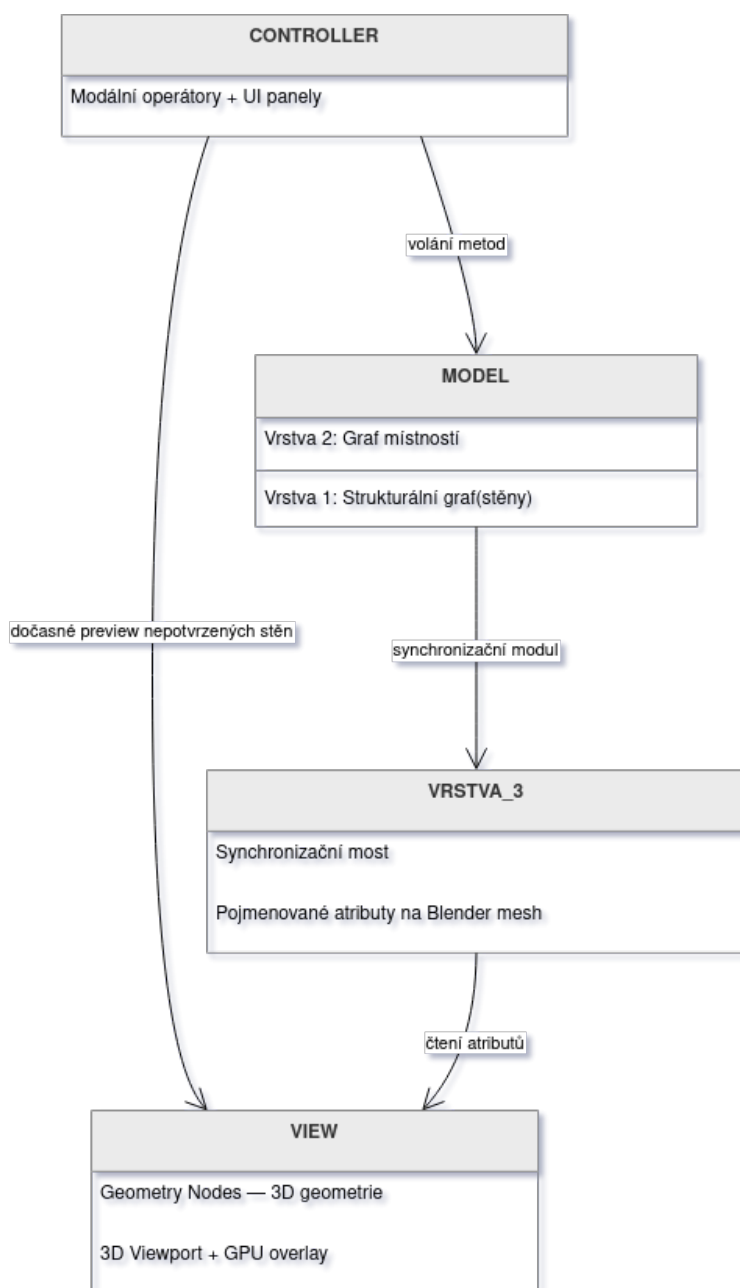
2.2.2.3 Vrstva 3: Synchronizační most

Vrstva 3 (synchronizační most) transformuje data z předchozích dvou vrstev do formátu, který Blender využívá pro zobrazení a další manipulaci s modelem. Datový tok je striktně jednosměrný: nejprve se plně zpracuje logika půdorysu a místností a teprve poté se tyto informace promítnou do výsledné geometrie. Tento postup zaručuje, že vizuální výstup vždy koresponduje s aktuálním stavem návrhu, čímž se předchází jakýmkoli rozporům mezi datovým modelem systému a tím, co reálně vidí uživatel.

2.2.3 Vzor MVC v kontextu Blenderu

Architektura přirozeně odpovídá vzoru `MVC` v prostředí Blenderu. **Model** tvoří Vrstvy 1 a 2 — čisté Python struktury bez závislosti na `bpy`, testovatelné izolovaně jednotkovými testy. **Vrstva 3** funguje jako synchronizační most, který zapisuje výsledky Modelu do Blender mesh. **View** zahrnuje Geometry

Nodes modifikátor pro 3D geometrii a GPU overlay pro kreslicí náhled a HUD. **Controller** jsou modální operátory zachytávající uživatelské vstupy a překládající je na volání metod Modelu; Controller nikdy nepíše přímo do Blender geometrie.



■ Obrázek 1 MVC diagram reprezentující oddělení vrstev addonu

2.2.4 Principy návrhu

Návrh se řídí pěti principy: **oddělení zájmů** (grafová logika nezávisí na bpy, lze ji testovat jednotkovými testy bez Blenderu); **nedestruktivní úpravy** (změna parametru spustí přegenerování přes Geometry Nodes, nikoli přepis geometrie); **zpětná vazba v reálném čase** (každá změna v Modelu se okamžitě projeví díky automatické reevaluaci GN modifikátoru); **modularita** (každý funkční požadavek FP1–FP7 je realizován v samostatném Python modulu); a **konvence Blenderu** (addon dodržuje standardní pojmenování operátorů, integraci Undo/Redo a strukturu addonu pro Blender Extensions).

2.2.5 Rozšiřitelnost architektury

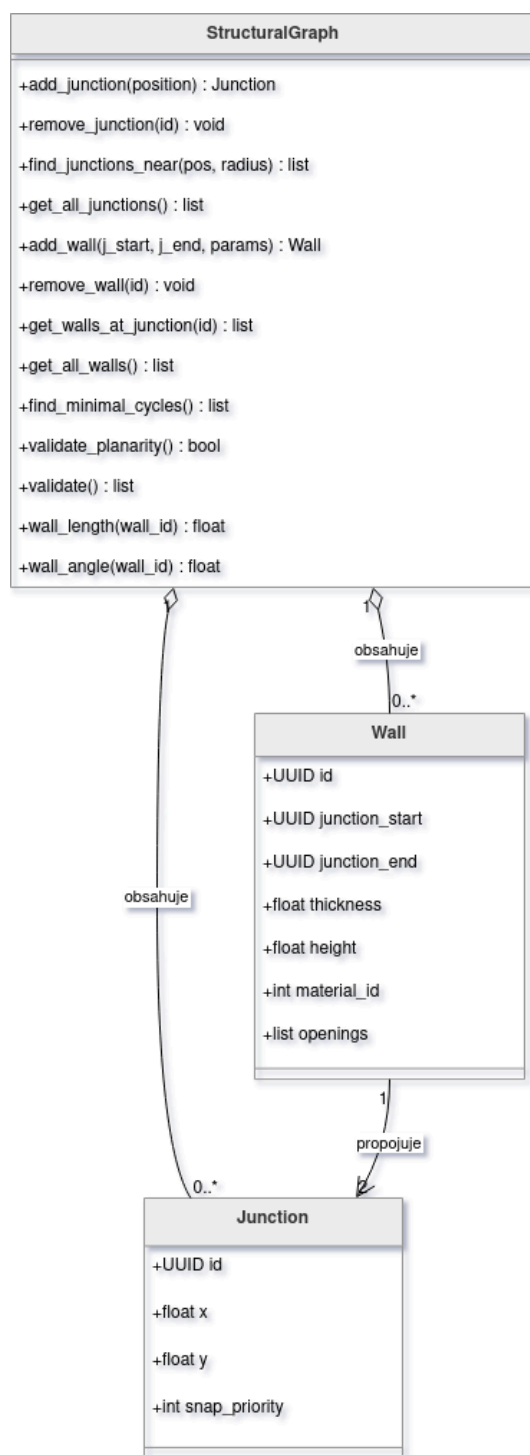
Třívrstvá architektura je navržena s výhledem na budoucí rozšíření. Více podlaží lze přidat koordinační vrstvou nad stávající Vrstvy 1 a 2 — ty samotné zůstanou beze změny. Nové typy prvků (sloup, příčka) nevyžadují změnu struktury grafu: stačí rozšířit sadu atributů a přidat odpovídající GN podstrom. Alternativní výstupní formáty pracují výhradně s daty Vrstvy 3, bez zásahu do zbytku systému.

2.3 Datový model

Architektura vymezila, z jakých vrstev se systém skládá a jak tyto vrstvy komunikují. Tato sekce přibližuje pohled na úroveň samotných dat: jaká je struktura entit, jaké atributy nesou a jaká pravidla musejí dodržovat. Datový model je záměrně oddělen od implementačních detailů — popisuje logické entity a jejich vztahy, nikoli konkrétní Python třídy s kódem.

2.3.1 Vrstva 1: Strukturální graf

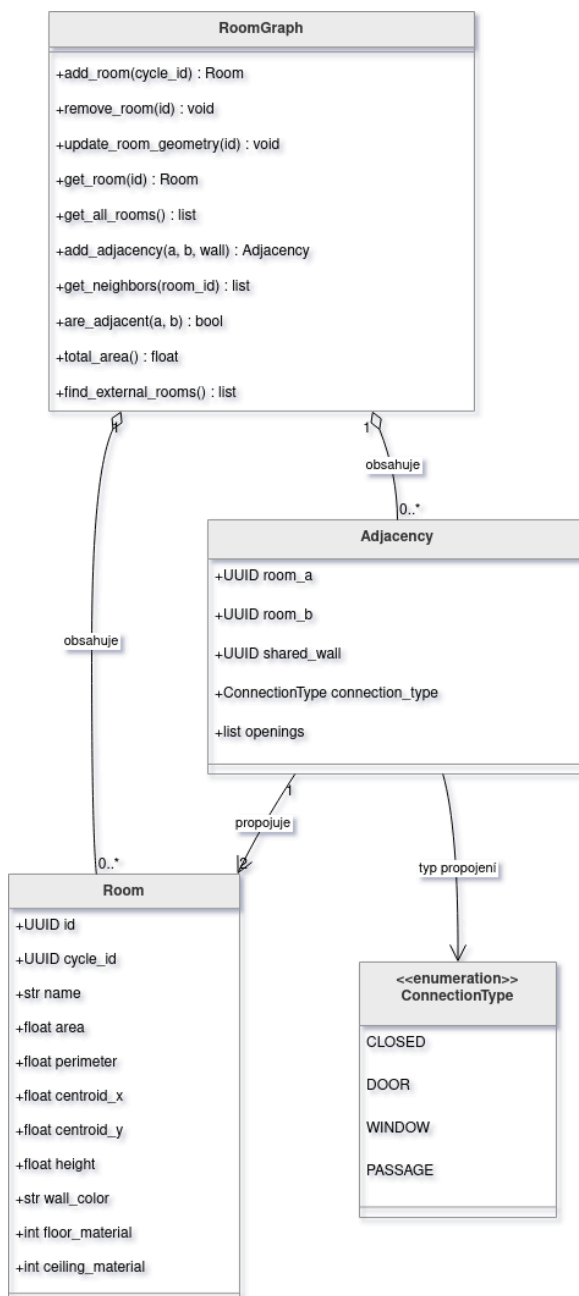
Třída `StructuralGraph` (Vrstva 1) nabízí operace pro celý životní cyklus grafu: přidávání a odebírání junctions a stěn, vyhledávání junctions v blízkosti zadané polohy (pro snap-on-junction), získávání stěn napojených na daný junction, detekci minimálních cyklů (implementováno přes NetworkX [14]) a validaci planarity.



■ Obrázek 2 Diagram tříd na vrstvě 1

2.3.2 Vrstva 2: Sémantický graf místností

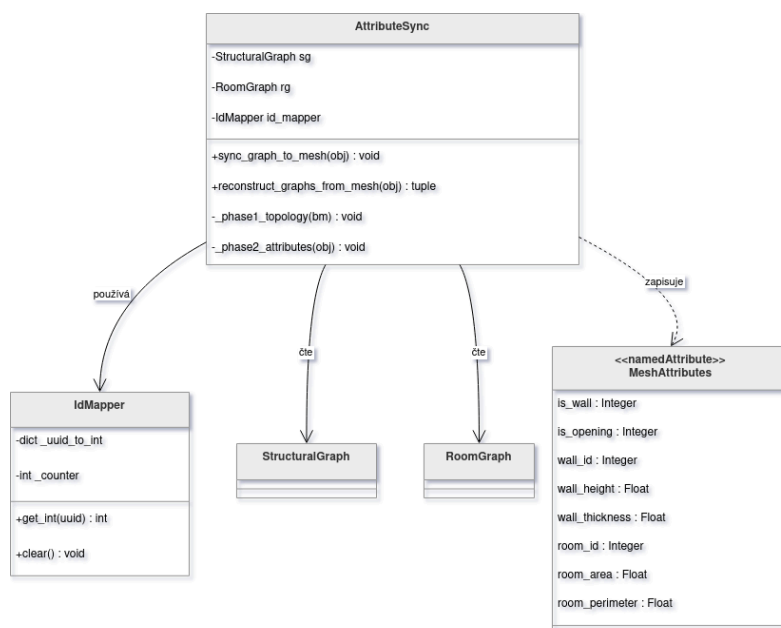
Třída `RoomGraph` spravuje kolekci místností a sousedností, poskytuje prostorové dotazy (sousedé místnosti, celková plocha dle typu, nalezení vnějších místností) a synchronizuje stav s Vrstvou 1 při každé topologické změně.



■ Obrázek 3 Diagram tříd na vrstvě 2

2.3.3 Vrstva 3: Synchronizační bridge

Vrstva 3 je jednosměrný datový kanál z Python grafů do Blender mesh. Synchronizační modul má dvě fáze, které musejí proběhnout v tomto pořadí: **fáze 1** udržuje topologii mesh v souladu s Vrstvou 1 (přidává a odebírá vrcholy, hrany a plochy); **fáze 2** zapisuje hodnotové atributy z Vrstev 1 a 2 na příslušné elementy mesh přes `mesh.attributes` [API](#). Geometry Nodes tyto atributy čtou jako pojmenované vstupy a generují z nich 3D geometrii.



■ Obrázek 4 Diagram tříd na vrstvě 3

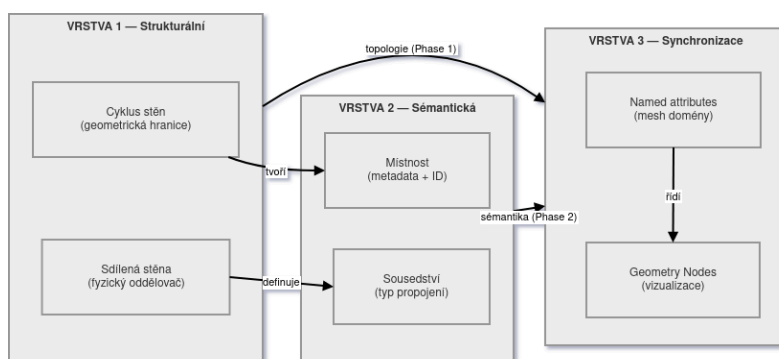
■ **Tabulka 10** Pojmenované atributy Vrstvy 3 zapisované synchronizačním modulem a čtené Geometry Nodes

Doména	Atribut	Typ	Účel
Vertex	junction_id	Integer	identifikace styku stěn
Edge	wall_id	Integer	identifikace stěny
Edge	wall_thickness	Float	tloušťka pro 3D generování
Edge	wall_height	Float	výška stěny
Face	room_id	Integer	identifikace místnosti
Face	room_area	Float	plocha místnosti v m^2
Face	is_wall	Integer	příznak stěnové plochy (1 = stěna, 0 = podlaha)
Face	is_opening	Integer	příznak plochy otvoru (cutter pro Mesh Boolean)

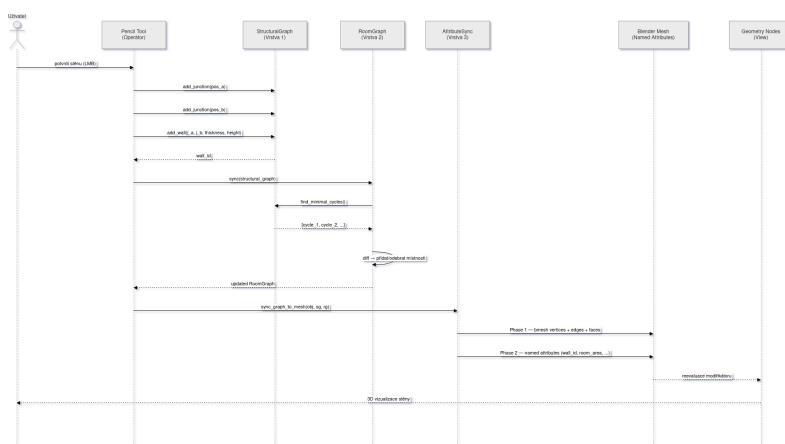
UUID identifikátory z Vrstev 1 a 2 se převádějí na celá čísla třídou `IdMapper` pro optimalizaci GPU zpracování v Geometry Nodes. Celoobjektová metadata a data persistence jsou ukládána jako Custom Property na Blender objekt.

2.3.4 Vztah mezi vrstvami

Všechny tři vrstvy jsou provázány jednosměrným asymetrickým tokem dat. Vrstva 1 (topologie) diktuje obsah Vrstvy 2 (sémantika): cyklus stěn ve Vrstvě 1 tvoří místnost ve Vrstvě 2; sdílená stěna dvou cyklů definuje sousedství. Vrstvy 1 a 2 společně zásobují Vrstvu 3 (synchronizace): topologie přechází z Vrstvy 1 do Vrstvy 3 v první fázi synchronizace; sémantická metadata přechází z Vrstvy 2 do Vrstvy 3 ve druhé fázi. Geometry Nodes čtou výsledné atributy a generují 3D geometrii. Zpětný tok neexistuje: Blender mesh je vždy odrazem aktuálního stavu grafů, nikoli jejich výchozím bodem.



■ Obrázek 5 Statický pohled — závislosti tříd



■ Obrázek 6 Dynamický pohled — sekvenční diagram

2.4 Návrh funkcí

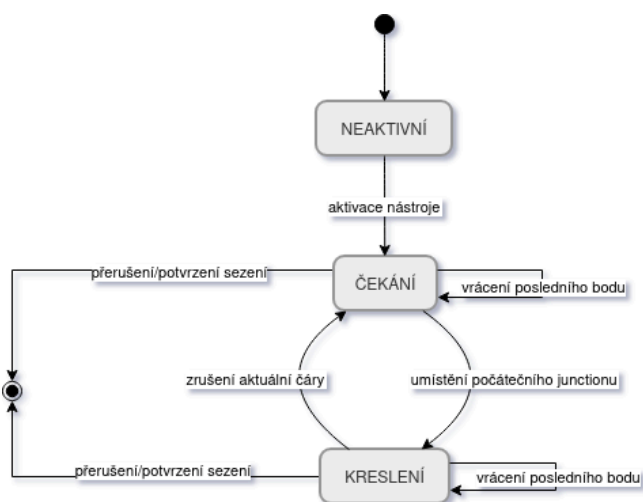
Architektura a datový model definují statiku systému — z čeho se skládá a jaká data nese. Tato sekce popisuje dynamiku: jak systém reaguje na uživatelské akce v každé funkci FP1 až FP7. Každá funkce je označena prioritou (must-have / should-have) v souladu s MVP filtrem z kapitoly 3.1.

2.4.1 FP1 — Nástroj tužka

Pencil Tool je primárním vstupním rozhraním addonu — modální operátor zachytávající vstupy myši a klávesnice a překládající je na operace nad Vrstvou 1. Veškerá logika kreslení probíhá ve 2D rovině XY; Z-souřadnice je ignorována.

2.4.1.1 Stavový automat

Operátor je řízen třístavovým automatem, který garantuje, že stěna nikdy nevznikne bez platného počátečního bodu a nelze skončit v nekonzistentním stavu. Stav **NEAKTIVNÍ** — nástroj je registrován, ale nepřijímá vstupy; jiné nástroje Blenderu fungují normálně. Stav **ČEKÁNÍ** — nástroj aktivní, kurzor sleduje myš, žádný počáteční bod ještě nebyl umístěn; **GPU** overlay zobrazuje existující geometrii a snap indikátory. Stav **KRESLENÍ** — první junction byl umístěn; overlay v reálném čase kreslí náhled stěny od posledního bodu ke kurzoru a **HUD** zobrazuje délku a úhel navrhované stěny. Z obou aktivních stavů lze sezení buď **potvrdit** (všechny stěny sezení se synchronizují do meshe a zapíší do Undo stacku jedním krokem) nebo **přerušit** bez uložení (všechny stěny a junctions sezení se odstraní, mesh zůstane nezměněn). Zrušení aktuální čáry (přechod KRESLENÍ → ČEKÁNÍ) neodstraňuje dříve potvrzené stěny téže sezení.



■ Obrázek 7 Stavový diagram nástroje tužka

2.4.1.2 Snapping

Snapping má přesně danou prioritní hierarchii. Nejvyšší prioritu má **snap na existující junction** (*must-have*) — kurzor blíže než 15 pixelů od existujícího

junctionu se přichytí na jeho přesné souřadnice. Nižší prioritu mají snap na osu, snap na mřížku a snap na úhel násobků 45° (*všechny should-have*). Aktivní snap je vizuálně indikován žlutým kruhem u kurzoru; podržení Shift snap dočasně potlačí.

2.4.1.3 Vizuální zpětná vazba (GPU overlay)

Veškerý kreslicí náhled probíhá v `draw_handler` registrovaném na `SpaceView3D` — neukládá se do geometrie ani datového modelu. Náhled stěny: čára od posledního junctionu ke kurzoru v odlišné barvě od potvrzených stěn. `HUD`: délka navrhované stěny a úhel k poslednímu úseku aktualizované při každém pohybu myši; ve stavu ČEKÁNÍ zobrazuje stavovou zprávu. Snap indikátor: barevný kruh u kurzoru při aktivním snapu. Náповěda kláves: ikony kláves a tlačítek myši v dolní stavové liště Blenderu (`STATUSBAR_HT_header`) — vzor přejatý z nativního Blender Knife Tool.

2.4.1.4 Způsob generování stěny

Preview stěny reprezentuje osu stěny, avšak fyzická stěna má tloušťku, kterou je nutné rozložit. Tři **módy generování stěny** (*should-have*) — Center (symetricky na obě strany), Left (tloušťka nalevo od směru kreslení) a Right (napravo) — odpovídají konvenci používané ve vektorových nástrojích a `CAD` programech. Mód lze přepínat klávesou `Tab` kdykoli během aktivního nástroje bez přerušování sezení; pojmy „levá“, a „pravá“ jsou vztaženy k *směru kreslení*, nikoli k pohledu kamery.

2.4.2 FP2 — Parametrické objekty a otvory

FP2 definuje, jak se změna parametru stěny propíše do geometrie a jak jsou otvory svázány se stěnou tak, aby se pohybovaly spolu s ní.

2.4.2.1 Parametry stěny a update mechanismus

Každá stěna ve Vrstvě 1 nese atributy `thickness`, `height` a `material_id`. Změna kteréhokoli z nich spustí přesně definovaný **update cyklus**: validace nové hodnoty → zápis do Vrstvy 1 → případný přepočít sousedních místností ve Vrstvě 2 → Vrstva 3 fáze 2 serializuje pouze změněný atribut → Geometry Nodes reevaluace. Tento update je záměrně levnější než přidání stěny: neprovádí se detekce cyklů ani fáze 1 sync, protože topologie mesh se nemění.

2.4.2.2 Otvory — GN Mesh Boolean

Otvory (dveře, okna) jsou svázány s konkrétní stěnou — uchovávají svou polohu, šířku, výšku a výšku parapetu jako součást záznamu o stěně ve Vrstvě

1. Výřez otvoru v geometrii zajišťuje Geometry Nodes prostřednictvím operace Mesh Boolean: ze zadaných rozměrů sestaví tvar výřezu a ten od stěny odečte. Výsledek se aktualizuje automaticky při každé změně parametrů.

2.4.2.3 Vložení pravoúhlé místnosti z parametrů

Toto vložení (*must-have*) je alternativní vstupní metodou k Pencil Toolu: uživatel zadá šířku, hloubku, výšku a tloušťku stěn v N-panelu a addon vytvoří čtyři junctions a čtyři stěny se středem v poloze 3D kurzoru Blenderu. Výsledná místnost je datově nerozeznatelná od místnosti nakreslené tužkou — všechny navazující operace (FP5, FP6, FP7) fungují identicky. V rámci MVP se místnost vkládá vždy jako samostatná izolovaná místnost bez sdílených junctions s existující sítí. Rozšiřující možností (*should-have*) této funkce, je zadání více parametrů pro vkládanou místností, např. počet stěn, jejich poměr nebo tvar místnosti.

2.4.3 FP3 — Detekce místností a metadata

FP3 (*must-have*) představuje klíčovou funkcionalitu, která odlišuje Floor-PlanMaster od běžných nástrojů pro 3D modelování: místnosti nevznikají manuálním zadáváním, ale jako implicitní výsledek nakreslené dispozice. Jakmile stěny vytvoří uzavřený prostor, systém jej automaticky detekuje jako místnost. Přepočet neprobíhá kontinuálně, ale efektivně pouze ve chvíli, kdy je to skutečně nutné — tedy po každé úpravě půdorysu. Odstraní-li uživatel dělicí stěnu mezi dvěma místnostmi, systém tyto prostory sloučí do jednoho a zachová přitom metadata původní místnosti.

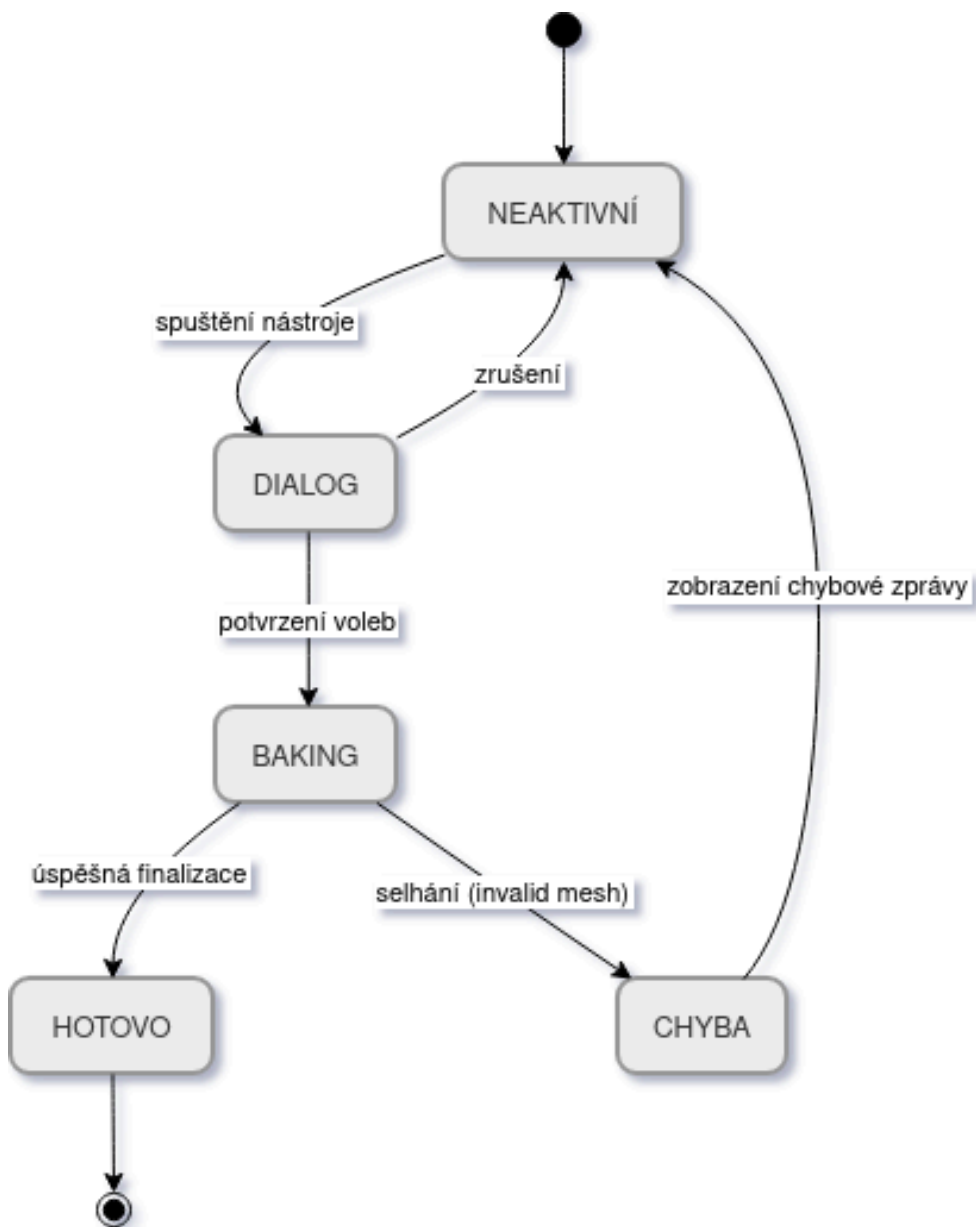
U každého prostoru systém průběžně eviduje jeho **plochu**, **obvod**, polohu středu (pro správné ukotvení textových popisků) a uživatelsky definovaný **název**.

Geometrie uložená v souboru modelu obsahuje veškerá potřebná data k tomu, aby systém při opětovném načtení projektu nebo po použití funkce Zpět kompletně zrekonstruoval aktuální stav dispozice. Uživatelské názvy místností se ukládají odděleně; pokud by z jakéhokoli důvodu chyběly, celková funkčnost logiky půdorysu zůstane zachována — místnosti budou existovat i nadále, pouze se zobrazí bez svých pojmenování.

2.4.4 FP4 — Finalizační nástroj

Finalizační nástroj (*must-have*) provede nevratný převod parametrického modelu do statické polygonové sítě vhodné pro export, UV mapování nebo herní engine. Jde o jednosměrnou operaci — proto addon před zahájením vygeneruje záchranný bod Undo. Operátor je řízen automatem se stavy NEAKTIVNÍ

→ DIALOG (spuštění) → BAKING (potvrzení voleb) → HOTOVO nebo CHYBA.



■ **Obrázek 8** Stavový diagram finalizačního nástroje

V dialogu uživatel volí: **organizaci výstupu** (jeden objekt / per místnost / separace stěny + podlahy + stropy), **přiřazení materiálů** (automaticky z metadat Vrstvy 2 nebo ponechat výchozí Blender materiál), **čistění atributů** (odstranit named attributes z výsledné sítě pro úsporu dat) a **zachovat originál** (duplikovat před finalizací vs. finalizovat přímo).

2.4.5 FP5 — Kontextová nabídka

Kontextová nabídka (*should-have*) poskytuje rychlý přístup k méně frekventovaným operacím (přejmenování, smazání, rozdělení stěny) bez nutnosti hledat je v panelech. Vyvolává se stiskem **RMB** ve 3D Viewportu — zaběhlá Blender konvence.

Klíčovým mechanismem je **raycast**: addon vrhne paprsek z pozice kurzoru přes Vrstvu 3 (Blender mesh s named attributes) a identifikuje typ elementu. Plocha → místnost; hrana → stěna; vrchol → junction; prázdný prostor → globální akce. Obsah nabídky se dynamicky přizpůsobuje kontextu: uživatel vidí vždy jen akce relevantní pro kliknutý prvek, nikoli celý katalog operátorů addonu.

Pro kontext místnosti jsou dostupné přejmenování, změna materiálu a smazání (kaskádové odebrání stěn z Vrstvy 1). Pro kontext stěny: editace tloušťky, výšky a materiálu; přidání otvoru; rozdělení stěny vložením nového junctionu; smazání. Pro kontext junctionu: smazání s přilehlými stěnami; sloučení s nejbližším junctionem (merge v toleranci). Pro prázdný prostor: přepnutí viditelnosti mřížky, kótování a spuštění Pencil Tool — alternativa ke klávesové zkratce pro uživatele preferující nabídky.

2.4.6 FP6 — Interaktivní manipulátory

Gizmos (*should-have*) jsou interaktivní grafické ovládací prvky ve 3D Viewportu, které umožňují přímou geometrickou manipulaci taháním myši bez přepínání nástrojů. Vzor je přejat z Archipack [5], kde výběr stěny automaticky zobrazí táhla pro tloušťku a výšku.

Addon definuje tři typy manipulátorů. **Manipulátor tloušťky stěny** (světle modrá) — obousměrná šipka kolmá na osu stěny v rovině XY; táhnutím se aktualizuje `wall_thickness` ve Vrstvě 1 a spouští fáze 2 sync a GN reevaluace, topologie Vrstvy 1 zůstává nezměněna. **Manipulátor výšky stěny** (zelená) — svislá šipka na středu stěny, pohyb omezen na osu Z; aktualizuje `wall_height` bez změny topologie. **Manipulátor pohybu junctionu** (žlutá) — kruh na vybraném junctionu; pohyb striktně omezen na rovinu XY (**2D zámek**) — Z-složka tahu je zahozena, protože pohyb mimo rovinu by narušil planaritu Vrstvy 1 a způsobil selhání detekce místností.

2.4.7 FP7 — Automatické kótování

Kótovací overlay (*should-have*) zobrazuje rozměry stěn a metriky místností průběžně ve 3D Viewportu bez nutnosti aktivovat speciální nástroj. Text je vykreslován jako `POST_PIXEL` overlay přes `draw_handler` registrovaný na

SpaceView3D ([12]) — zůstává čitelný nezávisle na úhlu kamery, protože pracuje v souřadnicích obrazovky. Data jsou čtena výhradně z Vrstev 1 a 2, nikoli z geometrie scény: délky stěn z Euklidovské vzdálenosti junction–junction, plochy a centroidy z uzlů Vrstvy 2. Viditelnost kótování přepíná globální přepínač v Nastavení (klávesa T).

2.5 Návrh uživatelského rozhraní

Architektura a funkce definují, co systém dělá. Tato sekce popisuje, jak uživatel se systémem komunikuje — kde jsou nástroje umístěny, jaká klávesová zkratka co spouští a jak viewport vizuálně reflektuje aktuální stav. Návrh UI vychází z principu konzistence s ekosystémem Blenderu: každý prvek rozhraní kopíruje vzor, který uživatelé Blenderu již ovládají, aby minimalizoval náklady na učení. Současně musí návrh reflektovat omezení Blender Python API zejména nemožnost přidat nativní pracovní mód, a dosáhnout ekvivalentního výsledku kompozicí dostupných mechanismů.

2.5.1 Toolbar — umístění nástroje tužka

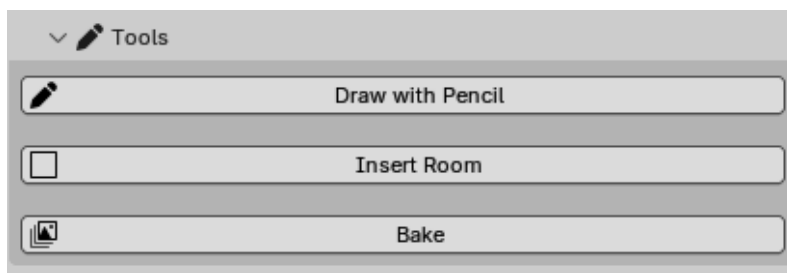
Konvencí Blenderu je, že nástroj ovládaný levým tlačítkem myši v kontinuálním modálním režimu patří do Toolbaru. Pencil Tool tuto podmínku splňuje: po aktivaci přebírá všechny vstupy a každé LMB kliknutí potvrzuje bod. Addon proto registruje Pencil Tool jako `WorkspaceTool` — Blender jej automaticky zobrazí jako ikonové tlačítko v Toolbaru, vizuálně zvýrazní při aktivaci a zobrazí tooltip s názvem a klávesovou zkratkou.

Při aktivaci nástroje se v levém horním rohu viewportu a v sekci Active Tool v N-panelu zobrazí ovládací prvky pro tloušťku, výšku a příchycení stěny — vzor přejatý z nativních sculpting nástrojů Blenderu.

2.5.2 Postranní panel (N-panel)

N-panel (Sidebar) je standardním místem pro trvalé rozhraní addonů. Addon přidává záložku **FloorPlanMaster** se čtyřmi skládacími sekcemi. Toto členění přejímá vzor z Archipack [5] kde je panel rozdělen na sekci operátorů a sekci parametrů vybraného objektu.

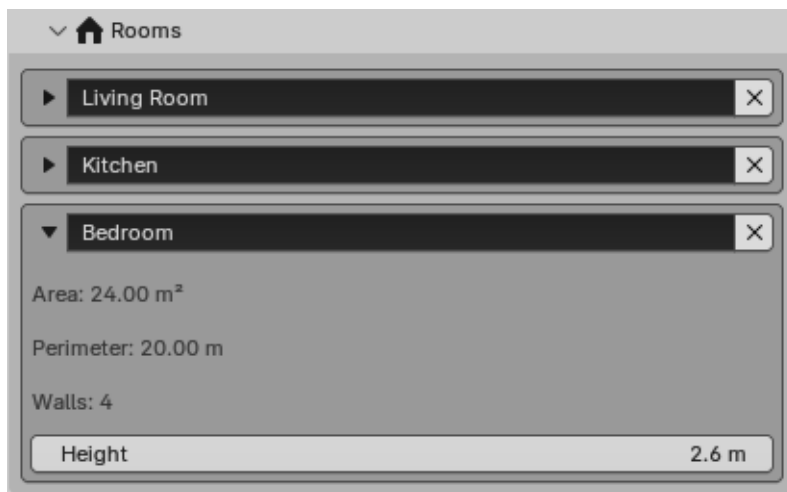
Sekce Nástroje sdružuje spouštěče akcí: tlačítko Pencil Tool (alternativa ke klávesové zkratce D), Vložit místnost (otevřít inline formulář s poli šířka, hloubka, výška, tloušťka) a Zapéct (spouští finalizační pop-over FP4 s volbami výstupu). Toto odlišení je záměrné: Blender konvencí je, že akce vkládající nové prvky patří do sekce Nástrojů, nikoli do sekcí modifikujících výběr.



■ Obrázek 9 Návrh UI pro sekci Nástroje

Sekce Místnosti je přehledem uzlů Vrstvy 2: seznam všech místností s editovatelným názvem a průběžně aktualizovanou plochou. Kliknutím na položku dojde k výběru místnosti ve viewportu a rozbalení detailního pohledu přímo pod položkou (obvod, výška, počet stěn). Vzor odpovídá technice „list panel s automatickým výběrem“, kterou Blender nativně používá pro vertex groups nebo shape keys.

Jako rozšiřující možností této sekce je přidat hierarchický list stěn dané místnosti pro přehlednou kontrolu uživatelem.



■ Obrázek 10 Návrh UI pro sekci Místnosti

Sekce Nastavení obsahuje globální parametry scény uložené v `Scene PropertyGroup`. Záměrně jsou sem zařazeny pouze parametry ovlivňující chování celého projektu — nikoli parametry jednotlivých prvků.



■ Obrázek 11 Návrh UI pro sekci Nastavení

■ Tabulka 11 Přehled globálních nastavení v sekci Nastavení N-panelu s výchozími hodnotami

Parametr	Výchozí hodnota	Popis
Systém jednotek	Metrický	Přepíná zobrazení rozměrů; interně vždy metry
Výchozí tloušťka stěny	0,3 m	Přednabídnuo při kreslení a vkládání místnosti
Kótovací overlay	Vypnuto	Zapíná draw_handler FP7
Barevné odlišení místností	Vypnuto	Každá místnost dostane automaticky odlišnou barvu
Názvy místností ve viewportu	Zapnuto	Text v centroidu místnosti
Highlight výběru	Zapnuto	Označí aktuálně vybraný prvek
Manipulátory (gizmos)	Vypnuto	Zobrazí táhla pro vybranou stěnu

Sekce aktuálně vybrané stěny je umístěna na spodu záložky a zobrazuje se výhradně tehdy, když je ve scéně vybrána stěna. Zobrazuje délku stěny, editovatelnou tloušťku a výšku a seznam otvorů s možností přidání a odebrání. Detail každého otvoru poskytuje editaci názvu, typu (dveře/okno), šířky, výšky, výšky parapetu a relativní polohy na stěně.

Možným rozšířením je přidání editace polohy stěny a jejích koncových bodů.

2.5.3 Klávesové zkratky

Návrh klávesových zkratk se řídí dvěma principy: zkratky musejí být kompatibilní se stávající svalovou pamětí uživatelů Blenderu a AutoCADu [9]; zkratky platné v modálním kontextu nesmějí kolidovat s globálními Blender zkratkami. Klávesový mód Pencil Tool je záměrně analogický Blender Knife Tool: rozlišuje stav ČEKÁNÍ a stav KRESLENÍ.

■ **Tabulka 12** Klávesové zkratky addonu s kontextem a zdůvodněním volby klávesy

Akce	Klávesa	Kontext	Zdůvodnění
Aktivovat Pencil Tool	D	Object Mode	D = Draw; AutoCAD konvence; v Blenderu Object Mode neobsazeno
Umístit bod / pokračovat	LMB	Pencil Tool aktivní	LMB jako primární potvrzovací vstup — Blender standard od v2.80
Potvrdit sezení	Enter	Oba stavy	Blender konvence pro potvrzení modálního operátoru (Knife Tool, Loop Cut)
Vrátit poslední bod	Z	Oba stavy	Z jako Undo; bezpečné — Blender Z (shading pie) blokováno po dobu aktivity operátoru
Zrušit aktuální čáru	RMB	Stav KRESLENÍ	Analogie s Knife Toolem; ve stavu ČEKÁNÍ RMB propustí do Blenderu (kontextová nabídka)
Přerušit sezení bez uložení	ESC	Oba stavy	Zruší všechny změny sezení; identické chování s Knife Tool a Loop Cut
Potlačit snap	Shift (držet)	Pencil Tool aktivní	Blender konvence pro dočasné přepnutí snap modu
Přepnout kótování FP7	T	3D Viewport	T = Toggle; alternativně nastavitelné v Preferences

2.5.4 FloorPlan kontext — simulovaný pracovní mód

Ideálním designovým řešením by byl vlastní pracovní mód paralelní k Object Mode a Edit Mode — „FloorPlan Mode“, s jednoznačně vymezeným kontextem, vlastními klávesami a explicitní bariérou vůči obecným Blender nástrojům. Blender Python [API](#) [\[11\]](#) však neumožňuje přidání nativního módu; `object.mode_set()` je pevně omezeno na módy implementované v C jádru. Návrh proto realizuje ekvivalentní řešení složením tří mechanismů:

1. **Interakční stav** — FloorPlan mód lze aktivovat pouze pro objekt, který je současně aktivní i vybraný. Aktivace je mapována na klávesu rezervovanou v Object Mode (např. `Shift+Q`). Po zapnutí se režim stává „vlastníkem“, interakcí v 3D Viewportu pro daný objekt. Deaktivace vrátí kontrolu obecným Blender nástrojům.
2. **Selektivní propuštění vstupů** — Během aktivního FloorPlan módu modální komponent řídí příjem klávesnice a myši. Navigace kamery (orbit/zoom/pan), Undo/Redo a nastavení panelů se propouští Blenderu. Levé kliknutí provádí sémantický výběr stěny či místnosti pomocí raycasting přes mesh. Běžné Blender zkratky pro editaci (W, G, Tab, X) jsou konzumovány addonem, aby se zabránilo nežádoucím operacím. Pokud Pencil Tool právě kreslí, vstup je jemu svěřen.
3. **Vizuální rozlišení kontextu** — Během aktivního módu se vypne standardní Blender highlight vybraných objektů a jejich místo zaujme sémantické zvýraznění prvků: oranžové obarvení vybrané stěny, průhledné vyplnění vybrané místnosti, barevné označení otvorů. Globální informační overlaye (délky stěn, plochy místností, výšky) zůstávají viditelné pro všechny FloorPlan objekty v projektu nezávisle na módu, ale interaktivní kontext — výběr a úpravy — patří pouze objektu, který je vlastníkem aktivního režimu.

Jeden `.blend` soubor tak může obsahovat více FloorPlan objektů. Informace všech jsou viditelné pro orientaci, ale editační operace proběhnou pouze pro jeden vybraný vlastníka módu. Přepnutí režimu na jiný FloorPlan objekt je jednoduché — deaktivace stávajícího a aktivace nového přes tutéž klávesovou zkratku.

Koncepční analogie s Edit Mode. FloorPlan mód sdílí filozofii s Blender Edit Mode: oba se aktivují na vybraném objektu a transformují dostupnou sadu akcí do speciálně vymezené domény. Edit Mode přesunuje fokus z úrovně objektu na úroveň primitivů (vrcholy, hrany, faces), kde se standardní operace (G, S, R) aplikují na geometrické prvky; FloorPlan mód analogicky přesunuje fokus do domény architektonického návrhu (junctions, walls, rooms), kde interakční gesta mají odlišný význam a dostupné nástroje jsou úzce zaměřeny

na kreslení a editaci půdorysu. Oba módy tedy sdílí paradigma — vstup se interpretuje v kontextu speciální domény, nikoli v obecném Object Mode kontextu. Tato analógie propůjčuje FloorPlan módu na sebe pochopitelnosti: uživatelé již zvyklí na mód-driven workflow v Blenderu rozpoznají princip a přenesou své mentální modely mezi módy. Výsledek je seamless přepínání mezi obecným 3D modelováním (Object Mode) a specifickým architektonickým návrhem (FloorPlan mód) bez pocitu diskontinuity.

2.6 Testování a validace návrhu

Před zahájením implementace je nezbytné ověřit, že navržená architektura a funkce společně spolehlivě pokrývají zadání z MVP scope a neobsahují logické mezery. Tato kapitola plní roli kontroly návrhu prostřednictvím kontrolních seznamů, teoretických průchodů scénáři a analýzy okrajových případů.

2.6.1 Pokrytí požadavků

Každý must-have prvek FP1–FP4 má jednoznačně přiřazenou návrhovou sekci (kapitola 3.4), datový model (kapitola 3.3) a průchoditelný tok dat napříč vrstvami. Should-have prvky FP5–FP7 jsou navrženy a datově pokryty, ale nejsou součástí MVP scope — architektura je připravena je pojmout bez změny Vrstev 1 a 2.

■ **Tabulka 13** Tabulka pokrytí požadavků návrhovou dokumentací; ✓ = součást MVP, — = odloženo za MVP

Požadavek	Navrhující sekce	Datový model	MVP
FP1 — modální kreslení	FP1 — stavový automat	Junction, Wall (L1)	✓
FP1 — snap na junction	FP1 — snapping	Junction.position (L1)	✓
FP2 — update stěny	FP2 — update mechanismus	Wall.thickness/height (L1)	✓
FP2 — otvory GN Boolean	FP2 — otvory	Wall.openings, L3 atributy	✓
FP3 — detekce místností	FP3 — detekce cyklů	Room (L2) z cyklu (L1)	✓
FP4 — finalizace	FP4 — interakce s modelem	L3 → statická mesh	✓
FP5 — kontextová nabídka	FP5 — raycast + akce	L3 raycast → L1/L2	—
FP6 — manipulátory	FP6 — typy gizmos	Wall.thickness/height, Junction.pos	—
FP7 — kótování	FP7 — datový pipeline	L1 délky, L2 area + centroid	—

Nefunkční požadavky jsou pokryty architekturně. NP1 (Geometry Nodes jako backend, Python jako manažer) je zajištěn oddělením Vrstev 1–2 od Vrstvy 3. NP2 (výkon a nedestruktivnost) je zajištěn jednosměrným tokem dat a dvoufázovou synchronizací — změna atributu nevyvolá detekci cyklů (pouze fáze 2). NP3 (použitelnost) je zajištěn výběrem klávesových zkratk a barevné sémantiky konzistentních s Blender konvencemi a validačními chybami hlášenými pop-overem.

2.6.2 Průchod scénáři použití

Průchod scénáři ověřuje, že navržená architektura a funkce společně pokrývají celý uživatelský workflow. Všechny scénáře (UC 1.1–3.2) jsou plně řešitelné must-have prvky (FP1–FP4).

- **UC 1.1 (hmotová studie z parametrů):** architekt zadá plochu a poměr stran → FP2 vytvoří čtyři stěny → Vrstva 2 vypočítá metriky (FP3) → opakování pro stavební program.
- **UC 1.2 (kreslení dispozice tužkou):** aktivace Pencil Tool (FP1) → kreslení s snapem na existující junctiony (FP1) → uzavření cyklu spouští detekci místnosti (FP3) → úprava parametrů v N-panelu (FP2) → GN reevaluace.
- **UC 2.1 (obkreslení 2D půdorysu):** vizualizátor vloží obrázek na pozadí → aktivuje Pencil Tool s snapem (FP1) → odklikává rohy podle obrázku → addon generuje stěny a detekuje místnosti (FP3) → nastavení tloušťky a výšky (FP2).
- **UC 2.2 (příprava pro rendering):** výběr stěny a přidání otvoru (FP2 — GN Boolean) → parametry otvoru v N-panelu → finalizace (FP4).
- **UC 3.1 (level blackout):** game designer aktivuje Pencil Tool (FP1) → kreslí sérii navazujících místností (FP1, FP3) → uniformní výška v N-panelu (FP2).
- **UC 3.2 (export herní úrovně):** přidání dveřních otvorů (FP2 — GN Boolean) → ověření ploch místností v N-panelu (FP3) → finalizace a export (FP4).

Následující tři scénáře rozšiřují základní workflow o pokročilé funkce a nejsou součástí MVP scope. Jejich implementace je záměrně odložena za prvotní fázi projektu. Architektura je však již připravena jejich pojetí bez jakýchkoli změn Vrstev 1 a 2, a datový základ (geometrické metriky a identifikace prvků) je dostupný od MVP.

- **UC 1.3 (kótování):** zapnutí kótovacího overlaye (FP7) — závisí na FP7, datový základ dostupný.
- **UC 2.3 (kontextová editace):** RMB na prvek → dynamická nabídka (FP5) — závisí na FP5, datový základ dostupný.
- **UC 3.3 (interaktivní manipulace):** gizmo pohybu junctionu (FP6) — závisí na FP6, datový základ dostupný.

2.7 Hodnocení návrhu uživatelského rozhraní

Kapitola 3.5 definovala návrh [UI](#) addonu. Před zahájením implementace je vhodné provést jeho systematické hodnocení — ověřit, že navržené rozhraní je konzistentní s ekosystémem Blenderu, splňuje klíčové principy použitelnosti a umožňuje novému uživateli dokončit základní úkoly bez zbytečných překážek. Hodnocení aplikuje tři doplňující se metody: kontrolu konzistence, heuristické hodnocení dle Nielsen a kognitivní průchody vybranými scénáři.

2.7.1 Konzistence s ekosystémem Blenderu

Kontrola konzistence hodnotí, jak plynule addon zapadá do stávajícího Blender ekosystému a zda je vnitřně soudržný.

Vnější konzistence. Pencil Tool je registrován jako WorkspaceTool v T-panelu, shodně s nativními modálními nástroji Knife Tool, Loop Cut a Poly Build — addony umísťující modální nástroje mimo Toolbar (Archimesh [\[4\]](#), Archipack [\[5\]](#)) tak činí z historických důvodů předcházejících WorkspaceTool [API](#). Klávesové zkratky respektují stávající Blender keymapping: LMB jako potvrzení, RMB jako kontextová nabídka, ESC pro zrušení modálního operátoru (identické s Knife Tool). Barevná sémantika přejímá konvence nativních nástrojů: oranžová pro výběr odpovídá standardnímu Blender theme, modrá pro nepotvrzené prvky odpovídá Loop Cut preview, žlutá pro snap odpovídá nativnímu Blender snap markeru. N-panel sekce Nástroje → Místnosti → Nastavení odpovídá logické hierarchii Blender Properties editoru (Tool → Item → View).

Vnitřní konzistence. Každá klíčová akce je dosažitelná více cestami se shodným výsledkem: aktivace Pencil Tool přes D, tlačítko v N-panelu i klik na ikonu v Toolbaru vede do identického vstupního stavu FP1 automatu; přepnutí kótování přes T, přepínač v kontextovém menu i v Nastavení mění tentýž stav FP7 draw_handleru. Obousměrná synchronizace výběru (klik na místnost v N-panelu vybere ji ve viewportu a naopak) zabraňuje situaci, kde uživatel edituje jiný prvek než zvýrazněný.

2.7.2 Heuristické hodnocení

Heuristické hodnocení porovnává návrh s třemi Nielsenovými heuristikami použitelnosti nejvíce relevantními pro nástrojový addon v 3D editoru.

Viditelnost stavu systému. Pencil Tool jako modální operátor s více stavy je nejvýraznějším rizikovým místem — uživatel musí vždy vědět, zda je nástroj aktivní a v jakém stavu se automat nachází. Návrh adresuje toto riziko na třech úrovních: HUD overlay v levém dolním rohu viewportu zobrazuje

aktuální stav automatu (ČEKÁNÍ / KRESLENÍ) a nápovědu platných kláves — vzor přejatý z Blender Knife Tool; dynamická preview stěna ve stavu KRESLENÍ potvrzuje, že první bod byl zaregistrován; snap indikátor (žlutý kruh) signalizuje výsledek příštího kliknutí ještě před jeho provedením — SketchUp [8] používá identický vzor. Hodnocení: heuristika je dobře pokryta.

Shoda se skutečným světem. Terminologie addonu (místnost, stěna, otvor, tloušťka, výška) odpovídá slovní zásobě architektů i game designérů. Rozměry jsou zobrazeny v nastaveném systému jednotek, nikoli jako interní hodnoty v metrech. Workflow Pencil Tool (bod → stěna → uzavření cyklu → místnost) odpovídá způsobu, jakým architekti ručně kreslí půdorysy — místnosti vznikají jako logický důsledek uzavřených smyček bez explicitní deklarace; zásadní rozdíl oproti Archimesh [4], kde se místnosti vkládají jako předpřipravené objekty. Hodnocení: heuristika je pokryta.

Uživatelská kontrola a svoboda. ESC kdykoli během Pencil Tool zruší aktuální kreslicí akci a vrátí automat do stavu ČEKÁNÍ; opakované ESC deaktivuje nástroj. Z vrátí poslední potvrzený junction bez opuštění nástroje — zkrácení iterativního cyklu o dva kroky oproti globálnímu Ctrl+Z. Každá operace modifikující Vrstvy 1 nebo 2 je zapsána do Blender Undo stacku. Nedestruktivní architektura GN zaručuje, že změna parametru stěny je kdykoliv vratitelná. Smazání místnosti nebo stěny z kontextové nabídky zobrazí pop-over s potvrzením — nevratné akce bez potvrzení by porušovaly jak tuto heuristiku, tak Blender konvenci. Hodnocení: heuristika je pokryta.

2.7.3 Kognitivní průchody

Kognitivní průchod simuluje zkušenost nového uživatele při plnění konkrétního úkolu. Hodnotitel prochází každý krok a klade čtyři otázky: ví uživatel, jakou akci provést? Vidí ovládací prvek? Pokročil správným směrem? Je zpětná vazba správná?

Poznámka: Hodnocení je expertní průchod architektury; kvalitativní uživatelské testování neproběhlo a je doporučeno v implementační fázi.

Zvoleny jsou dva reprezentativní scénáře:

UC 1.2 — Kreslení dispozice tužkou. Aktivace nástroje (krok 1) — ikona tužky v Toolbaru s tooltipem je dohledatelná průzkumem Toolbaru nebo N-panelu; po aktivaci se ikona zvýrazní a HUD zobrazí stavovou zprávu se stavem ČEKÁNÍ — zpětná vazba je správná. Umístění prvního bodu (krok 2) — stavová zpráva říká „LMB: umístit první bod,“; po kliknutí se preview linka táhne ke kurzoru a HUD přejde do stavu KRESLENÍ — zpětná vazba potvrzuje přechod stavu. Uzavření místnosti (krok 3) — snap indikátor u prvního junctionu signalizuje možnost uzavření; uzavření cyklu způsobí okamžité zobrazení barevné výplně plochy — vizuální signál detekce místnosti

bez technické hlášky. Identifikovaný potenciál rizika: uživatel nemusí vědět, že uzavření cyklu na první junction je podmínkou vzniku místnosti; snap indikátor a chybějící barevná výplň při neuzavření toto riziko zmírňují.

UC 1.1 — Vložení místnosti z parametrů. Nalezení funkce (krok 1) — tlačítko „Vložit místnost,“ je v sekci Nástroje jako první viditelný prvek; tooltip říká „Vloží pravoúhlou místnost se středem na 3D kurzoru“. Identifikované riziko: uživatel bez Blender zkušenosti nemusí znát pojem „3D kurzor,“ — riziko akceptovatelné, protože 3D kurzor je fundamentální Blender koncept předcházející práci s libovolným addonem pro modelování. Zadání parametrů a potvrzení (krok 2) — po potvrzení se místnost ihned zobrazí ve viewportu jako obarvená plocha a N-panel sekce Místnosti zobrazí novou položku s vypočítanou plochou — okamžitá vizuální a numerická zpětná vazba potvrzující správnost operace.

2.7.4 Závěr hodnocení

Hodnocení provedené třemi metodami neprokázalo žádné systémové nedostatky zabraňující zahájení implementace. Kontrola konzistence potvrdila soulad s Blender konvencemi na všech úrovních. Heuristické hodnocení dospělo u všech tří heuristik k pozitivnímu výsledku. Kognitivní průchody UC 1.2 a UC 1.1 identifikovaly dvě drobná rizika (nutnost uzavřít cyklus pro vznik místnosti; znalost pojmu 3D kurzor) a obě jsou zmírněna vizuálními mechanismy nebo jsou akceptovatelná pro cílovou skupinu se základní znalostí programu Blender.

Kapitola 3

Implementace

Kapitola implementace navazuje přímo na výsledky analýzy v kapitole [Kapitola 1](#) a technický návrh z kapitoly [Kapitola 2](#). Východiskem je [MVP](#) rozsah a třívrstvá architektura definovaná v sekci [Kapitola 2.2.2](#). Implementace zachovává stejný datový tok i rozdělení odpovědností: čisté Python jádro řeší topologii a sémantiku, synchronizační vrstva je most do programu Blender a vizualizaci zajišťuje [GN](#) modifikátor a [UI](#) vrstva.

3.1 Addon pro Blender

FloorPlanMaster se integruje do programu Blender jako Python balíček spuštěný přímo uvnitř Blender interpretu. Tato integrace je výrazně hlubší než pouhé přidání funkce: addon sdílí se softwarem stejný Python interpret, může registrovat vlastní typy, ovlivňovat chování viewportu a zachytávat události v reálném čase. Blender pro tuto formu integrace definuje dvojici funkcí — funkci pro aktivaci, která zaregistruje veškeré operátory, panely a draw handlers addonu, a funkci pro deaktivaci, která vše bez zbytků odregistrované. Životní cyklus addonu je tak plně řízen systémem programu; addon nezanechává žádné vedlejší efekty mimo dobu své aktivace.

Izolace kódu závislého na Blenderu od čistého Pythonu nevznikla z umělé v rámci návrhu, ale z bezprostřední praktické nutnosti. Jakmile byly napsány první testy pro strukturální graf, pytest okamžitě selhal: `import bpy` ve vrcholku modulu způsobil `ModuleNotFoundError`, protože mimo Blender Python interpret toto [API](#) neexistuje. Vzor s podmíněným importem byl proto zaveden hned na začátku jako obranný mechanismus, nikoliv jako pozdní refaktor. Příмым důsledkem je, že veškerá základní logika — topologické grafy, validátory, matematické utility — je dostupná jako standardní Python balíček a pokrytá automatickými testy bez jakékoliv závislosti na Blenderu. Do budoucna je možné toto oddělení udělat více elegantně a návrh i implementace s tím počítají.

Algoritmy pro detekci cyklů v planárních grafech jsou postaveny nad knihovnou NetworkX — Python balíčkem třetí strany, který Blender ve svém interpretu nenabízí. Pro distribuci jako Blender Extension (formát zavedený

v Blenderu 4.2) je NetworkX přibalen jako archiv ve formátu Python wheel přímo do balíčku addonu. Python dokáže z takového archivu importovat moduly přímo bez rozbalení. Vstupní bod addonu proto po spuštění přidá složku s archivem do vyhledávací cesty interpretu — NetworkX se stane dostupným bez jakéhokoliv zásahu uživatele.

3.2 Organizace modulů

Struktura složek a organizace modulů addonu reflektuje architekturu celého systému. Jednotlivé vrstvy jsou rozděleny dle svých logických a funkčních vlastností. Každý modul v `core/` a `utils/` smí importovat výhradně standardní Python knihovny a lokální moduly ze stejné úrovně — nikoliv Blender moduly. Porušení pravidla se projeví okamžitě jako selhání testovací sady, která se spouští standardním pytestem. Složky `operators/`, `ui/` a `geometry/` naopak na Blender spoléhají plně. Oddělení je záměrně struktiní, bez hybridních modulů — každý soubor patří buď do čistého Python světa, nebo do Blender světa, nikoliv do obou.

```
src/
├── __init__.py          # addon entry point
├── core/
│   ├── structural_graph.py # Vrstva 1 – junction, wall topologie
│   ├── room_graph.py      # Vrstva 2 – room, synchronizace
│   ├── sync.py            # Vrstva 3 – Python grafy → Blender mesh
│   ├── final_mesh_builder.py
│   ├── junction_solver.py
│   └── validators.py      # sdílené funkce s chybovými kódy
├── operators/           # Blender modální operátory (FP1–FP4)
├── ui/                  # N-panel, overlay, properties
├── geometry/            # Geometry Nodes tree builder
├── utils/
│   ├── constants.py      # výchozí hodnoty, validační limity
│   └── math_helpers.py   # 2D geometrie, polygon area, centroid
├── tests/               # pytest unit testy pro core/ a utils/
└── wheels/              # bundlované .whl závislosti (networkx)
```

3.3 Vrstvy

Třívrstvá architektura tvoří výpočetní jádro addonu. Porozumět jejímu fungování nejlépe umožňuje sledovat konkrétní událost: uživatel potvrdí nový vrchol v tužkovém nástroji a čeká, až se ve viewportu vytvoří nová stěna. Tato zdánlivě jednoduchá akce projde všemi třemi vrstvami a nakonec spustí reevaluaci `GN` modifikátoru v C++ jádru Blenderu.

3.3.1 Vrstva 1 — Strukturální graf

První vrstva přijme požadavek na přidání nového vrcholu a stěny a ještě než cokoliv zapíše do svého stavu, provede sadu validačních kontrol na vstupu: zda nová stěna není příliš krátká, zda na zadaných souřadnicích již neexistuje jiný vrchol, zda nevznikne duplicitní stěna. Pokud jakákoliv kontrola selže, vrstva požadavek odmítne a vrátí specifický chybový kód — grafová struktura zůstane beze změny v platném stavu. Toto pravidlo platí pro všechny operace zápisu.

Pro efektivní vyhledávání blízkých vrcholů — operaci klíčovou pro funkci přichycení v tužkovém nástroji — udržuje Vrstva 1 prostorový index v podobě slovníku mapujícího zaokrouhlené souřadnice na identifikátor existujícího vrcholu. Dotaz na vrcholy v blízkosti dané polohy je tak operací v konstantním čase, nezávislou na celkovém počtu vrcholů v grafu.

Topologicky nejnáročnější operací Vrstvy 1 je detekce minimálních cyklů — uzavřených stěnových smyček vymezujících potenciální místnosti. Implementace vychází z planárního embeddingu konstruovaného nad knihovnou NetworkX.

Planární embedding je reprezentace planárního grafu, která ke každému vrcholu ukládá cyklické pořadí jeho sousedů v rovině. Z tohoto pořadí lze algoritmicky odvodit hranice všech ohraničených oblastí — tzv. faces planárního grafu — aniž by bylo nutné provádět geometrické výpočty. Pro půdorys, jehož stěny tvoří planární graf, odpovídá každá ohraničená oblast právě jedné potenciální místnosti.

Z grafu jsou pak nejprve odstraněny listy, které součástí žádného cyklu být nemohou; poté se zkonstruuje planární embedding a z jeho hraniční struktury se extrahují všechny minimální ohraničené oblasti. Výsledkem je deterministická sada cyklů. Výsledek je uložen a invaliduje se pouze při změně topologie — průběžné dotazy na seznam cyklů se tak nemusejí přepočítávat při každém dotazu nebo úpravě od uživatele.

Nejpodstatnější algoritmický problém Vrstvy 1 se projevil až při testování T-spojů a X-spojů. Prvotní implementace funkce pro výpočet rohů stěnového quadu procházela všechny sousední stěny v daném vrcholu a vybírala průsečík postranní přímky stěny s nejmenším absolutním parametrem $|t|$ podél přímky — kde t je skalární parametr parametrické rovnice přímky $P(t) = A + t \cdot \vec{d}$ udávající vzdálenost průsečíku od referenčního bodu A ve směru $\vec{d}$. Pro L-spoje (dva sousedé ve vrcholu) tato strategie fungovala korektně. Jakmile se ale vrchol spojoval se třemi nebo více stěnami, $|t|$ minimum vybíralo špatnou stranu — rohový bod quadu přeskočil přes sousední stěnu a vytvořil překrývající se geometrii. Oprava vyžadovala přepracování výpočtu na angular-sort algoritmus: `_junction_entries()` seřadí všechny stěny v daném vrcholu podle

úhlu jejich odchozího směru, čímž vznikne cyklus sousedů. Levý roh stěny se poté hledá jako průsečík levé postranní přímkou stěny i s pravou postranní přímkou angulárně-sousední stěny $i + 1$ v CCW pořadí. Tento přístup je deterministický pro libovolný počet stěn ve vrcholu.

3.3.2 Vrstva 2 — Graf místností

Vrstva 2 reaguje na dokončení operace ve Vrstvě 1 a synchronizuje svůj stav s aktuálním seznamem minimálních cyklů. Tato synchronizace neprobíhá průběžně při každé dílčí změně, ale vždy až po dokončení celé uživatelské operace. Synchronizace porovná aktuální sadu cyklů s množinou stávajících místností na základě kanonického klíče každého cyklu — setříděné množiny jeho hraničních stěn. Nové smyčky dostanou nové objekty místností; smyčky, které z topologie zmizely, jsou odstraněny spolu se svými metadaty; zachované smyčky zůstávají nedotčeny — jejich jméno, typ a ostatní atributy jsou stabilní.

Tato volba není nahodilá: alternativou bylo sledovat identitu cyklu přes pořadí vrcholů nebo přes první stěnu smyčky, ale obě alternativy jsou nestabilní při geometrických editacích. Přidání nové stěny do smyčky může změnit pořadí vrcholů i první stěnu při traversalu. Konkrétní příklad: místnost tvoří čtyři stěny S_1, S_2, S_3, S_4 a uživatel ji pojmenuje „Obývací pokoj„. Pokud by byl klíč místnosti určen pořadím vrcholů traversalu $[J_1, J_2, J_3, J_4]$, pak přidání nové příčky kdekoli jinde v půdorysu — zcela mimo tuto místnost — může traversal planárního embeddigu projít vrcholy v jiném pořadí $[J_2, J_3, J_4, J_1]$, čímž by vznikl jiný klíč a místnost by ztratila přiřazené jméno. Setříděná množina UUID $\{S_1, S_2, S_3, S_4\}$ je naopak neměnná vůči pořadí traversalu: dokud se nemění hraniční stěny místnosti, klíč zůstává stejný bez ohledu na zbytek půdorysu. Tato volba má přímý dopad na uživatelský zážitek: pojmenování a metadata místnosti „přežijí“ topologicky nesouvisející úpravy, které by jiným přístupem způsobily ztrátu identity.

3.3.3 Vrstva 3 — Synchronizace s Blenderem

Vrstva 3 překládá Python datový model do formátu, jemuž Blender a Geometry Nodes rozumějí. Jejím výstupem je jeden Blender mesh objekt — tzv. base mesh půdorysu — nesoucí topologii i parametry ve formě pojmenovaných atributů čitelných přímo z [GN](#) stromu.

Plná synchronizace probíhá ve dvou fázích. V první fázi se mesh rekonstruuje od nuly: pro každou stěnu se vypočítá přesný čtyřúhelníkový obrys v rovině XY pomocí `junction_solver.py` (zmíněný angular-sort nad stěnami v každém vrcholu), aby spoje fungovaly pro libovolné úhly bez mezer a přesahů.

Na vrcholech se třemi a více stěnami se doplní výplňový polygon spoje a pro každý otvor se vytvoří šestistěnný cutter box. Současně se dopočte a vloží podlahový polygon místnosti po vnitřní hraně stěn.

Ve druhé fázi se přes `mesh.attributes` zapíše per-face atributy: `is_wall`, `is_opening`, `wall_id`, `wall_height`, `wall_thickness`, `room_id`, `room_area` a `room_perimeter`. Tím je zaručeno, že hodnoty jsou ihned dostupné pro Geometry Nodes. Po zápisu se objekt pouze označí pro přepočítání (`update_tag`) a reevaluace proběhne až při redraw; synchronizace záměrně nevolá explicitní synchronní update depsgraphu.

Implementace Vrstvy 3 prošla v průběhu vývoje jedním zásadním přepisem. Ten se týkal strategie generování stěnové geometrie. Prvotní implementace generovala stěny jako per-edge instance 3D kvádrů (`MeshToPoints` na hranách $\rightarrow$ `InstanceOnPoints`) s parametry rotace a škálování z atributů. Problémem bylo, že kvádry mají 90° čela, která neumožňují geometricky přesné spoje pro libovolné úhly: na T-spojích a X-spojích se kvádry překrývaly. Přejít na strategii *quad polygon + extrude* — kde každá stěna je 2D čtyřúhelník vypočítaný z průsečíků postranních přímků sousedních stěn — odstranil problém překryvů strukturálně.

3.3.4 Geometry Nodes — generování 3D geometrie

`GN` modifikátor přebírá od Vrstvy 3 připravený base mesh a autonomně generuje finální trojrozměrnou geometrii. `GN` strom je sestaven při první aktivaci addonu a nese interní číslo verze; při nesouladu verze se strom automaticky přebuduje. Tok zpracování ve stromu probíhá ve třech krocích: oddělení stěnových faces (`is_wall == 1`), oddělení opening cutter faces (`is_opening == 1`) a vytážení stěn podle per-face `wall_height`.

Kritický problém GN fáze přišel s implementací dveřních otvorů. EXACT Boolean solver (GN uzel Mesh Boolean DIFFERENCE/EXACT) vyžaduje watertight cutter mesh, jehož geometrie nesmí zasahovat mimo těleso, do kterého řeže. Dveřní cutter začínal na $z = 0$ — přesně na spodní ploše stěny. Numerická tolerance EXACT solveru tento stav vyhodnotila jako průsečík s podlahou stěny, což vedlo k nestabilnímu chování: díra buď nevznikla vůbec, nebo solver zobrazil cutter jako solidní těleso. Okenní otvory přitom fungovaly bezchybně, protože jejich cutter začíná na výšce nadpraží ($z = \text{sill_height} - \epsilon > 0$) — celý cutter leží uvnitř stěnového solidu. Oprava pro dveře spočívala v posunutí dolní hrany cutteru na $z = +0,01$ m (konstanta `OPENING_CUTTER_Z_OVERSHOOT`), čímž se vyloučila numerická interakce s podlahou. Výsledná mezera 1 cm u podlahy dveří je v praxi nepozorovatelná a pro účely addonu zanedbatelná. Do budoucna by bylo ideální problém vyřešit

exaktním výřezem a opravou geometrie, aby při práci nevznikaly nežádoucí artefakty.

3.4 Funkce

Funkce addonu jsou implementovány jako Blender operátory — spustitelné příkazy registrované u Blenderu a přiřazené klávesovým zkratkám nebo tlačítkům panelu. Každý operátor pracuje výhradně přes rozhraní datových vrstev a tvoří tak řídicí vrstvu (Controller) MVC architektury popsané v návrhu v sekci [Kapitola 2.2.2](#). Operátory nestojí na sobě navzájem — každý je samostatnou jednotkou, přistupující ke sdílené cache grafů a sdílenému stavu výběru přes definovaná rozhraní.

3.4.1 FP1 — Nástroj tužka

Vývoj tužkového nástroje byl doprovázen sérií propojených problémů, z nichž většina se projevila teprve při testování s reálnou dispozicí obsahující přibližně čtyřicet stěn. Prvním a nejzávažnějším byl výkonnostní problém (Issue 28): každý potvrzený vrchol spouštěl plnou synchronizaci Vrstvy 3 — přepočít stěnových obrysů, zápis osmi named atributů, reevaluaci GN modifikátoru a serializaci grafů do JSON custom property. S rostoucím počtem stěn cena každého kliknutí rostla lineárně, přičemž celková cena za nakreslení sekvence W stěn dosahovala $O(W^2)$. Řešením bylo přesunutí plného syncu na konec celé kreslicí sekvence a náhrada per-klik vizualizace čistě Pythonovými výpočty stěnových obrysů bez volání bpy. Cena per-klik klesla na $O(W)$; GN evaluace proběhne jednou při ukončení nástroje.

Druhým problémem bylo spuštění více instancí modálního operátoru (Issue 1): stisknutí klávesy D při aktivním tužkovém nástroji spustilo novou instanci, která zaregistrovala duplicitní GPU handlers a začala přetahovat události z první instance. Oprava vyžadovala guard na začátku `invoke()`: pokud je `_pencil_state is not None`, operátor okamžitě vrátí `{'CANCELLED'}`.

Třetím problémem byla obnova pohledu po deaktivaci (Issues 19, 30). Prvotní implementace obnovovala uloženou matici pohledu bezpodmínečně — pokud uživatel v průběhu kresby ručně otočil viewport a ukončil nástroj z perspektivy, pohled se nechtěně přeskočil zpět na uloženou horní ortografii. Oprava podmínila obnovu pohledu: matice se obnoví pouze tehdy, pokud viewport stále zobrazuje horní ortografii, jinak se zachová aktuální pohled uživatele.

3.4.2 FP2 — Výběr a parametrické úpravy

Výběr stěn kliknutím do viewportu prošel dvěma zcela odlišnými implementacemi (Issue 7). Prvotní přístup projektoval pozici myši na rovinu $Z=0$ a testoval, zda bod leží uvnitř 2D quadu stěny. Tato metoda fungovala v ortografickém top-down pohledu, ale selhávala v perspektivním pohledu nebo při pohledu šikmém: kliknutí na horní hranu 3D stěny se na rovině $Z=0$ projektovalo mimo stěnu — do středu místnosti. Přejít na screen-space testování šesti ploch 3D tělesa stěny (top, bottom, čtyři boky) situaci vyřešil: každá plocha se projektuje do 2D souřadnic obrazovky a klik se testuje vůči výsledným 2D polygonům. Tato metoda funguje v libovolném pohledu a přirozeně zohledňuje hloubku.

Přidávání otvorů přineslo nejsložitější validační logiku celého addonu. Otvor nesmí přesáhnout délku stěny, nesmí zasahovat do junction inset zóny a nesmí se překrývat s jiným otvorem. Hodnoty se průběžně clampují v metodě `check()` Blender dialogu. Při implementaci se ukázalo, že pořadí clampingů rozhoduje: pokud se nejprve clampuje šířka a pak poloha, pohyb středu otvoru ke kraji může neočekávaně zúžit otvor. Správné chování — výška je fixní, sill se zastaví u stropu, šířka je konstanta stěny nezávislá na poloze — vyžadovalo přesnou specifikaci invariantů ještě před psaním kódu clampingové logiky.

3.4.3 FP3 — Metadata místností

Implementace metadat místností odhalila specifický problém persistence jmen přes Undo a reload. Jméno místnosti žije v Python objektu v paměti (`Room.name`), který Undo/Redo neobnovuje — Blender při Undo obnoví mesh, ale Python grafy v `_graph_store` zůstanou ve stavu po poslední operaci. Výsledkem je, že přejmenování místností provedené před Undo krokem může po Undo zůstat zachováno, i když topologie se vrátila o krok zpět. Řešením je psaní jmen místností do JSON custom property při každém syncu a jejich rekonstrukce z JSON při každém `reset_graphs_for_obj()`. Tím je mesh vždy autoritativní zdroj pravdy a Python objekty jsou pouze odvozená cache rekonstruovatelná z meshe.

3.4.4 FP4 — Finalizace a bake

Finalizační operátor překonává architektonické oddělení procedurálního a statického modelu. Parametrický FloorPlan objekt nese celou historii topologie v JSON a vizualizaci deleguje na GN modifikátor; statický mesh je naopak prostá geometrická data bez sémantiky. Před samotným bake operátor rekonstruuje grafy z aktuálního stavu objektu, vygeneruje finální geometrii přes

`final_mesh_builder.py` s geometricky přesnými spoji a odstraní GN modifikátor i JSON custom property. Ochrana vstupu do Edit Mode zabraňuje situaci, kdy by uživatel neúmyslně přepsal GN mesh ručními editacemi: handler zachytávající přechod do mesh editace nabídne tři volby — zrušit přechod, provést bake a pak editovat, nebo vstoupit a ztratit sémantická data. Tato ochrana brání tichým chybám, které by jinak vznikly použitím standardní Blender editace na procedurálním objektu.

3.4.5 Neimplementované funkce

V aktuální verzi zůstávají neimplementované kontextové menu (FP5), 3D manipulátory (FP6) a automatické kótování (FP7). Export jako samostatný cílový výstupní krok nad rámec bake workflow také není dokončen. Architektura je však navržena tak, aby šlo tyto části doplnit jako izolované operátory bez zásahu do datového jádra.

3.5 Uživatelské rozhraní

Architektura **UI** — N-panel jako centrum interakce, overlay manager jako centrální dispatcher **GPU** vizualizace vzorem Mediátor a sdílený stav výběru jako modulová proměnná bez vazby na Blender vlastnosti — byla realizována dle návrhu v sekcích **Kapitola 2.5**. Tato kapitola popisuje implementační rozhodnutí a rozšíření, která vznikla nad rámec návrhu.

3.5.1 N-panel

Při implementaci N-panelu vyvstala otázka, kde persistovat vizuální stav rozhraní — konkrétně stav rozbalení záznamu místnosti. Modul-level Python slovník je smazán při reloadu addonu; přiřazení do `bpy.types.Scene` by sdílelo stav přes všechny FloorPlan objekty ve scéně, čímž by rozbalení místnosti v jednom projektu ovlivnilo zobrazení v jiném. Správným nosičem se ukázal samotný FloorPlan objekt jako Blender datový blok: custom property `room_expanded_{room_id}` je automaticky součástí `.blend` souboru, přežívá reload a je součástí Undo stacku — uživatel může `Ctrl+Z` obnovit nejen geometrii, ale i stav rozbalení panelu.

3.5.2 Overlay manager

GPU overlay architektura prošla zásadním refaktorem (Issue 38). Původní implementace registrovala `draw_handler_add` přímo v každém modulu a operátoru: pencil tool, select wall i room selection měly každý vlastní handler.

Výsledkem byl nekontrolovaný počet handlerů bez centrálního cleanup mechanismu; při reloadu addonu nebo při výjimce v `unregister()` mohly handlers přetrvávat a způsobit přístup do neplatné paměti při dalším redraw. Centrální overlay manager registruje jediný `POST_VIEW` a jediný `POST_PIXEL` handler, které dispatchují do listů registrovaných layer callbacků. Přidání nové overlay vrstvy vyžaduje vytvoření jednoho souboru v `ui/overlays/` a jedno volání `register_layer()` v `register()` — bez jakékoliv změny v handler logice.

Praktickým důsledkem refaktoru bylo oddělení persistentních vrstev (wall selection highlight, room highlight, labels) od transientních vrstev (pencil tool kreslení preview). Transientní vrstvy se registrují v `invoke()` a odregistrují v `_finish()` — bezpečně i při abnormálním ukončení operátoru. Odolnostní mechanismus `_report_layer_failure()` zajišťuje, že výjimka v libovolné vrstvě způsobí jednorázový log do konzole, ale nerozhodí zbytek overlay systému. Oproti návrhu přibýly dvě vrstvy, které nebyly explicitně navrženy: `wall_opening_highlight.py` (pasivní vizualizace hrany stěn a barevné rozlišení otvorů nezávisle na výběru) a `active_floorplan_hint.py` (orientační hint pro scény s více FloorPlan objekty).

3.5.3 Sdílený stav výběru

Prvotní implementace uchovávala UUID vybrané stěny jako `StringProperty` v `FloorPlanSettings` (Scene PropertyGroup). Tato volba se ukázala jako chybná ze dvou důvodů. Za prvé, `StringProperty` je persistovaná v `.blend` souboru a po Undo/Redo může ukazovat na UUID, které již neodpovídá aktuálnímu stavu grafu. Za druhé, programatické přiřazení UUID v select operátoru spouštělo PropertyGroup callback, který inicioval sync — kruhová závislost zastavitelná pouze příznakovým guardem `_updating_wall_props`. Přesun UUID do module-level `SelectionState` singletonu oba problémy odstranil: UUID žije výhradně v Pythonu mimo Blender property systém, nevyvolává callbacky a je vždy konzistentní s aktuálním stavem grafů.

`SelectionState` byl nad rámec návrhu rozšířen o dvě pole. Pole `object_name` uchovává název FloorPlan objektu, ke kterému výběr patří — bez něj by overlay vrstvy a `poll()` funkcí panelů nemohly rozlišit, zda vybraná stěna patří aktivnímu, nebo jinému FloorPlan objektu ve scéně. Pole `room_viewport_selected` rozlišuje výběr kliknutím ve viewportu od pouhého rozbalení záznamu v N-panelu — `poll()` top-panelu vlastností místnosti zobrazí panel pouze při kliknutí ve viewportu, aby procházení seznamu místností v N-panelu panel samovolně neposouvalo.

3.6 Současná omezení implementace

Tato kapitola shrnuje oblasti, které nebyly součástí **MVP** rozsahu, nebo kde implementace odhalila technické omezení, jehož oprava by vyžadovala strukturální zásah. U každé oblasti je uveden i rozsah potřebných doimplementací, aby byl výchozí bod pro navazující vývoj konkrétní.

Transformace FloorPlan objektu. Přesunutí nebo otočení FloorPlan objektu klávesami G, R ve Blenderu má neočekávaný efekt: při příští synchronizaci Vrstvy 3 jsou vrcholy meshe přepsány z L1 souřadnic, které o transformaci neví, a geometrie se vizuálně vrátí do původní lokální polohy. Problém vznikl tím, že souřadnice vrcholů se zapisují přímo jako lokální souřadnice objektu bez aplikace `matrix_world`. Architektura na opravu připravena je: stačí transformovat každou L1 souřadnici inverzí `obj.matrix_world` při zápisu do bmesh v `sync.py`. Pohyb junctions v operátorech musí naopak transformovat souřadnice myši z world space do lokálního prostoru. Žádná strukturální změna datového modelu není potřeba.

Duplikování FloorPlan objektu. Příkaz Shift+D zkopíruje custom properties i serializovaný JSON graf a addon pro duplicitní objekt rekonstruuje nezávislé Python grafy se shodnými UUID. V `_graph_store` tak existují dva grafy se stejným UUID prostorem, což může způsobit kolize při operacích odvolávajících se na UUID napříč objekty. Navíc Blender sdílí datový blok meshe mezi oběma objekty do explicitního unlinkování: synchronizace jednoho přepíše mesh, který vidí oba. Oprava vyžaduje dvě doimplementace: přegenerování UUID při rekonstrukci grafu z kopírovaného JSON a vynucené odlinkování datového bloku meshe ihned po detekci duplikátu.

Přejmenování FloorPlan objektu. Přejmenování objektu klávesou F2 změní `obj.name` v Blenderu, ale `_graph_store` zůstane indexovaný pod starým jménem. Žádný handler na přejmenování není registrovaný, takže sémantický mód se tiše přeruší: operátory nenajdou grafy pod novým klíčem a chovají se, jako by objekt neměl žádnou historii. Oprava je lokalizovaná a nevyžaduje zásah do datového modelu: buď přejít na klíčování podle stabilnějšího identifikátoru (`obj.data.name`), nebo registrovat `bpy.app.handlers.depsgraph_update_post` handler, který porovná uložené jméno s aktuálním a přeindexuje příslušný záznam.

Paralelní práce s více FloorPlan objekty. V sémantickém módu může být aktivní nejvýše jeden FloorPlan objekt najednou: `_mode_object_name` je jednoduché module-level pole a `SelectionState` drží jediné `object_name`. Tato vlastnost je záměrná a odpovídá **MVP** prioritě. Addon s více FloorPlan objekty ve scéně v ostatních ohledech počítá: `_graph_store` udržuje grafy pro všechny z nich a pasivní overlay vrstva `wall_opening_highlight.py` vizualizuje stěny a otvory každého viditelného FloorPlan objektu. Rozšíření

na plnohodnotnou podporu by vyžadovalo přepracování `_mode_object_name` a `SelectionState` z jednoduché hodnoty na slovník nebo sadu a úpravu `find_floorplan_obj(context)` — datový model ani vrstvy 1–3 žádnou úpravu nepotřebují.

Persistence stavu módu a výběru. Po načtení souboru je sémantický mód vždy vypnutý a výběr prázdný. `_mode_object_name` a `SelectionState` jsou module-level Python proměnné mimo Blender undo stack a nejsou serializovány do `.blend` souboru — jde o záměrné a bezpečné výchozí chování. Uživatel po otevření souboru začíná v neutrálním stavu a aktivuje mód explicitně (Shift+Q). Geometrie, grafy a veškerá topologická data jsou naopak plně perzistovaná: JSON custom property `_floorplan_graphs` je součástí `.blend` souboru a grafy se z ní korektně rekonstruují po každém načtení.



Kapitola 4

Testování

Závěr

TODO

Bibliografie

- [1] Blender Foundation, „Blender – Open Source 3D Creation Suite". 2024.
- [2] Architizer, „Architecture 101: What Is Parametric Architecture?". 2023.
- [3] ViBIM Global, „BIM vs CAD: Key Differences and When to Use Each". 2024.
- [4] A. Vazquez, „Archimesh – Architectural Objects for Blender". 2024.
- [5] S. Leger, „Archipack – Architectural Design Addon for Blender". 2024.
- [6] IfcOpenShell Contributors, „BonsaiBIM – Native IFC Authoring Platform for Blender". 2024.
- [7] International Organization for Standardization, „Industry Foundation Classes (IFC) for data sharing in the construction and facility management industries – Part 1: Data schema". 2024.
- [8] Trimble Inc., „SketchUp". 2024.
- [9] Autodesk Inc., „AutoCAD". 2024.
- [10] Autodesk Inc., „Revit". 2024.
- [11] Blender Foundation, „Blender Developer Documentation". 2024.
- [12] Blender Foundation, „Blender Python API Documentation". 2024.
- [13] S. Hu, W. Wu, Y. Wang, B. Xu, a L. Zheng, „GSDiff: Synthesizing Vector Floorplans via Geometry-enhanced Structural Graph Generation", *arXiv preprint arXiv:2408.16258*, 2024.
- [14] A. A. Hagberg, D. A. Schult, a P. J. Swart, „Exploring Network Structure, Dynamics, and Function using NetworkX", in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, G. Varoquaux, T. Vaught, a J. Millman, Ed., 2008, s. 11–15.

Contents of the attachment

└─ TODO TODO